



MÓDULO PROYECTO

Ciclo Superior Desarrollo de Aplicaciones Web

Departamento: Informática y Comunicaciones

IES “María Moliner”

Curso: 2025/2026

Grupo: S2I

Proyecto: APLICACIÓN WEB DE MENSAJES EFÍMEROS - SHOOTSTARS

Félix San José Hernán

Email: fsanjosehernan@hotmail.com

Tutor individual: María José González García

Tutor colectivo: Enrique Carballo Albarrán

Fecha de presentación: 27 de mayo 2026

ÍNDICE

1. DESCRIPCIÓN GENERAL DEL PROYECTO.....	4
1.1. Objeto del Proyecto.....	4
1.2. Antecedentes y Estado Actual.....	5
1.2.1. El paradigma de la red social tradicional.....	5
1.2.2. El movimiento de la mensajería efímera.....	6
1.2.3. Marco legal: RGPD y Privacidad por Diseño.....	6
1.3. Objetivos del Proyecto.....	7
Objetivos principales.....	7
Objetivos secundarios.....	7
1.4. Cuestiones Metodológicas.....	8
1.5. Entorno de Trabajo.....	9
1.5.1. Stack tecnológico — Modelo cliente-servidor.....	9
1.5.2. Herramientas de desarrollo.....	9
2. DESCRIPCIÓN GENERAL DEL PRODUCTO.....	10
2.1. Visión General del Sistema.....	10
2.2. Límites del Sistema.....	10
2.3. Funcionalidades Básicas.....	11
Gestión de usuarios.....	11
Gestión de mensajes.....	11
Interacciones.....	11
Administración.....	12
2.4. Tipos de Usuario y Proceso de Autenticación.....	12
2.5. Sistemas con los que Interactúa.....	13
2.6. Despliegue de la Aplicación.....	13
2.6.1. Plataforma tecnológica del servidor de producción.....	13
2.6.2. Proceso de instalación resumido.....	14
3. MARCO TEÓRICO Y TECNOLÓGICO.....	15
3.1. Visual Studio Code.....	15
3.2. Apache HTTP Server y mod_rewrite.....	15
3.3. PHP 8.2 – Hypertext Preprocessor.....	16
3.4. MySQL.....	17
3.5. Composer.....	18
3.6. PHPMailer y el Protocolo SMTP.....	19
3.7. HTML5 y la Canvas API.....	19
3.8. CSS3: Variables, Glassmorphism y Animaciones.....	20
Variables Custom (Custom Properties).....	20
Glassmorphism.....	21
Animaciones @keyframes.....	21
3.9. JavaScript ES6+ y la Fetch API.....	22

3.10. Seguridad Web: OWASP Top 10.....	22
3.11. Let's Encrypt y Certbot.....	23
4. PLANIFICACIÓN Y PRESUPUESTO.....	24
4.1. Planificación Temporal y Cronograma.....	24
4.2. Análisis de Riesgos.....	25
4.3. Presupuesto del Proyecto.....	26
4.3.1. Costes de personal.....	26
4.3.2. Costes de hardware (amortización).....	26
4.3.3. Costes de software.....	26
4.3.4. Costes de alojamiento.....	27
4.3.5. Gastos generales.....	27
4.3.6. Resumen presupuestario.....	27
4.4. Estimación COCOMO Básico.....	27
5. DOCUMENTACIÓN TÉCNICA.....	28
5.1. Diagrama de Casos de Uso.....	28
CU-01: Inicio de Sesión con 2FA.....	29
CU-02: Publicar un Mensaje.....	29
CU-03: Reaccionar a un Mensaje.....	30
CU-04: Administración de Contenido Reportado.....	30
5.2. Especificación de Requisitos Funcionales.....	31
RF-01: Módulo de autenticación.....	31
RF-02: Módulo de mensajes.....	31
RF-03: Módulo de interacciones.....	32
RF-04: Módulo de administración.....	32
5.3. Especificación de Requisitos No Funcionales.....	32
RNF-01: Seguridad.....	32
RNF-02: Rendimiento.....	33
RNF-03: Usabilidad.....	34
RNF-04: Portabilidad.....	34
5.4. Diseño de la Base de Datos.....	34
5.4.1. Modelo Entidad/Relación (E/R).....	34
5.4.3. Tablas y esquema SQL.....	35
5.4.4. Diccionario de datos.....	36
5.5. Diseño de la Interfaz de Usuario.....	38
5.5.1. Sistema de diseño "Space & Nebula".....	38
5.5.2. Diagrama de pantallas y flujo.....	39
5.6. Diseño de la Aplicación: API Interna.....	39
5.7. Implementación.....	41
5.7.1. Entorno de desarrollo.....	41
5.7.2. Estructura del código.....	41
5.7.3. Orientación del código.....	42

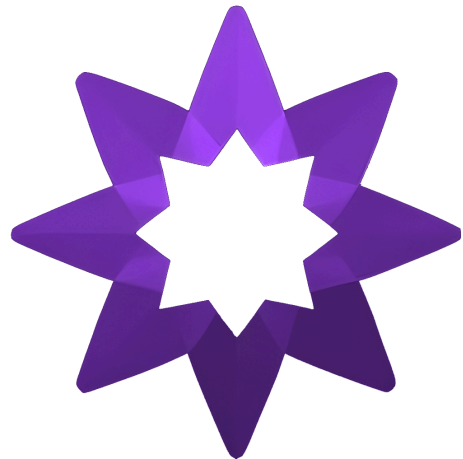
5.7.4. Cuestiones de diseño e implementación reseñables.....	43
5.7.5. Fragmentos de código implementados.....	45
5.8. Pruebas.....	50
5.8.1. Pruebas funcionales con diferentes dispositivos y sistemas operativos...	50
5.8.2. Pruebas de seguridad.....	51
5.8.3. Pruebas de compatibilidad con distintos sistemas y navegadores.....	52
5.8.4. Métricas de rendimiento y tiempos de ejecución.....	53
6. MANUALES DE USUARIO.....	53
6.1. Manual de Instalación.....	53
6.1.1. Requisitos básicos para la instalación.....	53
6.1.2. Proceso de instalación paso a paso.....	54
6.2. Manual de Usuario.....	58
6.2.1. Pantallas, ítems y flujo entre pantallas.....	58
7. CONCLUSIONES Y POSIBLES AMPLIACIONES.....	62
7.1. Conclusiones.....	62
7.1.1. Grado de cumplimiento de los objetivos.....	62
7.1.2. Dificultades encontradas.....	63
7.1.3. Grado de satisfacción.....	64
7.2. Posibles Ampliaciones.....	64
A corto plazo (sin cambios arquitecturales).....	64
A medio plazo (nuevas funcionalidades).....	65
A largo plazo (cambios arquitecturales).....	65
8. BIBLIOGRAFÍA.....	66
Documentación técnica oficial.....	66
Seguridad web.....	66
Diseño, accesibilidad y rendimiento web.....	66
Webgrafía.....	67
9. RELACIÓN DE FICHEROS EN FORMATO DIGITAL.....	67
ANEXO I – GLOSARIO.....	68
ANEXO II – HOJA DE ESTILOS CSS (main.css): EXTRACTOS REPRESENTATIVOS.....	71
A2.1. Reset y Fondo Global.....	72
A2.2. Sistema de Animaciones @keyframes.....	73
A2.3. Sistema de Botones.....	73
A2.4. Responsividad (Media Queries).....	75
A2.5. Sistema de Formularios con Validación Visual.....	76
ANEXO III – VALIDACIÓN DEL FORMULARIO DE REGISTRO.....	77

1. DESCRIPCIÓN GENERAL DEL PROYECTO

1.1. Objeto del Proyecto

El presente proyecto tiene como objeto el diseño, desarrollo e implementación de "ShootStars", una aplicación web centrada en la mensajería efímera y anónima. En una era digital donde la huella digital es cada vez más difícil de borrar, ShootStars propone una alternativa donde la privacidad y la fugacidad de la información son los pilares fundamentales.

La metáfora conceptual es la de una estrella fugaz: un mensaje que surge, brilla por un instante ante los ojos de otro usuario anónimo y desaparece. Esta filosofía contrasta radicalmente con el modelo de las grandes redes sociales, que monetizan la acumulación indefinida de datos personales del usuario.



ShootStars ofrece las siguientes funcionalidades principales:

- Publicar mensajes cortos de texto denominados "Estrellas" (hasta 500 caracteres), visibles para otros usuarios de forma completamente aleatoria, sin posibilidad de dirigirlos a un destinatario concreto.
- Interactuar mediante reacciones emoji de nueve tipos: me gusta (👍), risa (😂), triste (😞), enfado (😡), asco (🤢), sorpresa (😱), rezar (🙏), calavera (💀) y corazón (❤️). Control de unicidad: cada usuario solo puede reaccionar una vez por mensaje.
- Enviar respuestas anónimas denominadas "Ecos Estelares" (hasta 150 caracteres), asociadas al mensaje original sin revelar la identidad del remitente al público general.
- Autenticación de Doble Factor (2FA) mediante código OTP de 6 dígitos enviado al correo electrónico con PHPMailer y plantilla HTML personalizada con la estética del proyecto.
- Sistema de moderación comunitaria: los reportes acumulados (umbral: 5) generan notificación automática por correo electrónico al administrador. Panel

de administración con acciones de eliminación, bloqueo de usuario e ignorar reportes.

- Gestión de perfil: avatar personalizable con validación de tipo MIME real mediante la extensión finfo de PHP, historial de mensajes propios con paginación, edición y eliminación inline.

La aplicación está desplegada de forma pública y operativa en la URL <https://shootstars.sytes.net>, alojada en un servidor MiniPC propio con Debian GNU/Linux 12 (Bookworm), accesible desde internet mediante redirección de puertos y DDNS con el servicio No-IP.

1.2. Antecedentes y Estado Actual

1.2.1. El paradigma de la red social tradicional

Las grandes plataformas de redes sociales actuales —Twitter, Facebook, Instagram, TikTok— comparten un modelo de negocio fundamentado en la acumulación de datos del usuario. Cada publicación, reacción e interacción queda registrada de forma permanente, alimentando algoritmos de publicidad dirigida. Este modelo plantea problemas de profundo calado ético y social:



- Ausencia del derecho al olvido: una vez publicado un contenido, su eliminación completa es prácticamente imposible dado que puede haber sido capturado, indexado o replicado por terceros en cualquier momento.
- Construcción de identidades digitales permanentes: el usuario es incentivado a mantener una "marca personal" que persiste indefinidamente, generando presión social, autocensura y ansiedad digital.
- Opacidad algorítmica: los algoritmos priorizan el contenido más polarizante o que genera mayor engagement, con efectos demostrados negativos sobre la salud mental, especialmente en adolescentes.

- Monetización de los datos personales: la información del usuario —intereses, ubicación, relaciones, comportamientos— es el producto real en estos ecosistemas.

1.2.2. El movimiento de la mensajería efímera

En respuesta a estos problemas surgió Snapchat en 2011, revolucionando la comunicación digital con el concepto de mensajes que se autodestruyen. Aunque Snapchat evolucionó hacia un modelo más complejo, el concepto de lo efímero permeó el ecosistema: Telegram añadió mensajes autodestructibles, WhatsApp incorporó mensajes temporales, Instagram popularizó las Stories de 24 horas.



Sin embargo, todas estas soluciones presentan limitaciones comunes: están atadas a ecosistemas móviles propietarios, se centran en la comunicación entre contactos conocidos y mantienen registros de metadatos incluso cuando el contenido desaparece. Ninguna ofrece la aleatoriedad radical de ShootStars, donde el usuario no sabe quién leerá su mensaje ni cuándo.

1.2.3. Marco legal: RGPD y Privacidad por Diseño

El Reglamento General de Protección de Datos (RGPD, Reglamento UE 2016/679, artículo 17) reconoce el derecho al olvido: el derecho de las personas a solicitar la supresión de sus datos personales cuando ya no sean necesarios. El concepto de Privacidad por Diseño (Privacy by Design, artículo 25 del RGPD) postula que la privacidad debe incorporarse en los sistemas desde su concepción. ShootStars aplica este principio en múltiples capas: mensajes eliminables en cualquier momento con cascada de datos asociados, ecos anónimos, credenciales protegidas con hash irreversible y ausencia de rastreo entre usuarios.

1.3. Objetivos del Proyecto

Los objetivos se articulan en cinco ejes, diferenciando entre objetivos principales y secundarios:

Objetivos principales

- Seguridad robusta: implementar autenticación con contraseñas hasheadas en BCrypt (PASSWORD_DEFAULT en PHP) y sistema 2FA con OTP de 6 dígitos enviado al correo electrónico del usuario mediante PHPMailer/SMTP. El registro incluye validación de política de contraseña fuerte mediante expresión regular y verificación AJAX en tiempo real de disponibilidad de usuario y email.
- Experiencia de usuario de calidad: diseñar una interfaz moderna con temática espacial —fondo animado de estrellas en canvas HTML5, efecto glassmorphism en paneles, fuentes Orbitron y Outfit, degradado radial morado/negro— que funcione correctamente en escritorio y dispositivos móviles. La aplicación obtiene una puntuación de 99/100 en Rendimiento y 100/100 en Accesibilidad de Google Lighthouse.
- Privacidad por diseño: arquitectura donde los ecos sean anónimos para el público, los mensajes puedan eliminarse con eliminación en cascada de datos asociados, y no existan mecanismos de seguimiento cruzado entre usuarios más allá del estrictamente necesario.

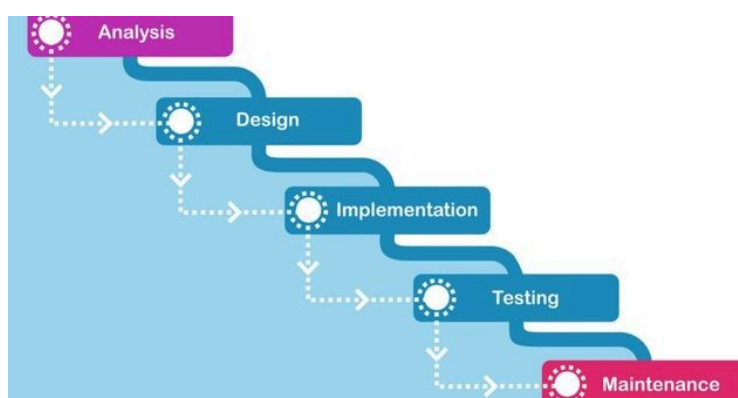
Objetivos secundarios

- Moderación comunitaria: sistema de reportes para mensajes y ecos con notificación automática al administrador al superar el umbral de 5 reportes, y panel de administración con acciones diferenciadas.
- Rendimiento y buenas prácticas: Vanilla JavaScript puro sin dependencias de frameworks para maximizar el rendimiento. Composer para gestión de dependencias, variables de entorno en .env, y 100% de consultas SQL con sentencias preparadas.

1.4. Cuestiones Metodológicas

Para el desarrollo de ShootStars se ha adoptado una metodología en Cascada (Waterfall), la más apropiada para un proyecto académico individual con requisitos bien definidos desde el inicio. La elección frente a metodologías ágiles (Scrum, Kanban) se justifica por:

- Proyecto unipersonal sin cliente externo que demande cambios continuos.
- Naturaleza secuencial natural del desarrollo web: no tiene sentido implementar antes de diseñar, ni probar antes de implementar.
- Requisitos estables y bien acotados desde la fase inicial.



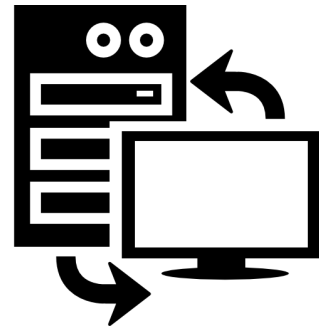
El modelo Cascada utilizado sigue el siguiente flujo secuencial con posibilidad de retorno al paso anterior si se detectan errores de planteamiento:

Fase	Periodo	Duración	Actividades principales
1. Análisis	Septiembre 2024	4 semanas	Definición de requisitos, estudio de referentes del sector (Snapchat, Whisper, Secret), reunión con tutora individual.
2. Diseño	Octubre 2024	4 semanas	Modelo E/R, esquema relacional, DDL SQL, wireframes de pantallas, diseño del sistema visual, definición de la API interna.
3. Implementación	Nov–Dic 2024	8 semanas	Configuración del servidor LAMP/Debian, desarrollo del backend PHP, maquetación HTML/CSS, integración JS con Fetch API, 2FA con PHPMailer.
4. Pruebas y Despliegue	Enero 2025	4 semanas	Pruebas funcionales, de seguridad y de compatibilidad. Bugfixing. Despliegue en producción. Redacción de la memoria.

1.5. Entorno de Trabajo

1.5.1. Stack tecnológico — Modelo cliente-servidor

ShootStars sigue un modelo cliente-servidor distribuido (aplicación web con servidor remoto), en el que el servidor Apache aloja la lógica de negocio PHP y la base de datos MySQL, mientras que el cliente (navegador web) se encarga de la presentación:



- Backend: PHP 8.2 (lenguaje de servidor), Apache2 con mod_rewrite (servidor HTTP), MySQL/MariaDB con motor InnoDB (base de datos), Composer (gestor de dependencias PHP).
- Frontend: HTML5 semántico, CSS3 (variables custom, flexbox, animaciones @keyframes, glassmorphism), JavaScript ES6+ Vanilla (sin frameworks externos).
- Librería externa: PHPMailer 6.11, única dependencia PHP del proyecto, instalada y gestionada mediante Composer.
- Servidor de producción: MiniPC con Debian GNU/Linux 12 (Bookworm). Acceso externo mediante DDNS con No-IP (subdominio shootstars.sytes.net) y port forwarding en el router doméstico.

1.5.2. Herramientas de desarrollo

- Visual Studio Code: IDE principal, licencia MIT (software libre). Extensiones utilizadas: PHP Intelephense (análisis estático PHP 8.2), Prettier (formateo automático), DotENV (resaltado de sintaxis para ficheros .env).
- Git y GitHub: control de versiones distribuido, licencia GPL v2. Repositorio público en <https://github.com/darks1g/ShootStars> con 34 commits registrados.
- MySQL Workbench: cliente gráfico para el diseño del esquema E/R y la ejecución de consultas SQL. Licencia GPL v2.
- Brave Browser y Mozilla Firefox: navegadores de referencia para pruebas, utilizando DevTools (inspector de elementos, consola JavaScript, pestaña Network para análisis de peticiones AJAX). Licencias libres.



- Google Lighthouse: herramienta de auditoría web integrada en las DevTools de Chrome/Brave. Permite evaluar Rendimiento, Accesibilidad, Buenas Prácticas y SEO de forma automática.

2. DESCRIPCIÓN GENERAL DEL PRODUCTO

2.1. Visión General del Sistema

ShootStars es una aplicación web de mensajería efímera y anónima de acceso libre. Su propuesta de valor fundamental es la aleatoriedad radical: al contrario que las redes sociales convencionales, los mensajes no se dirigen a seguidores ni a contactos conocidos, sino que son recibidos por usuarios aleatorios desconocidos. Esto libera la expresión de las presiones sociales habituales.

El sistema opera de forma autónoma como una aplicación web independiente, sin dependencias de servicios de terceros salvo el servidor SMTP de Gmail para el envío de correos del sistema 2FA. No forma parte de un sistema más global ni interactúa con otras aplicaciones externas en su versión actual.

2.2. Límites del Sistema

ShootStars es una aplicación autónoma con los siguientes límites bien definidos:

- El sistema gestiona exclusivamente el ciclo de vida de mensajes textuales cortos: creación, distribución aleatoria, interacción mediante reacciones y respuestas, moderación y eliminación.
- No implementa mensajería directa entre usuarios específicos ni sistema de "seguir" a otros usuarios.
- No almacena medios audiovisuales (imágenes en mensajes, vídeos, audio). Los únicos ficheros binarios que gestiona son los avatares de perfil de los usuarios.
- La única integración externa es el servidor SMTP de Gmail para envío de correos del sistema 2FA y notificaciones de moderación. Esta integración es unidireccional (solo envío).
- El sistema no implementa búsquedas ni filtros de mensajes. La distribución es estrictamente aleatoria.

2.3. Funcionalidades Básicas

Las funcionalidades del sistema se organizan por colecciones de datos:

Gestión de usuarios

- Registro de nuevos usuarios con validación de unicidad de nombre de usuario y email, y política de contraseña fuerte.
- Autenticación en dos fases: verificación de credenciales + OTP por correo electrónico.
- Recuperación de contraseña mediante token criptográfico enviado al email.
- Actualización de avatar con validación de tipo MIME real.
- Eliminación de cuenta con borrado en cascada de todos los datos asociados.

Gestión de mensajes

- Publicación de mensajes (1-500 caracteres). Validación con `mb_strlen()` para soporte de emojis.
- Distribución aleatoria: el endpoint `get_random_msg.php` devuelve un mensaje aleatorio visible mediante `ORDER BY RAND() LIMIT 1`.
- Historial de mensajes propios con paginación (`page`, `limit`, `offset`).
- Edición y eliminación de mensajes propios con verificación de propiedad.

Interacciones

- Reacciones de nueve tipos. Control de unicidad por usuario registrado (campo `id_usuario`) y por usuario invitado (cookie de navegador generada con `bin2hex(random_bytes(16))`), válida durante 1 año).
- Ecos (respuestas anónimas): creación, carga con paginación (`limit/offset`) y opción "Cargar más ecos".
- Reportes de mensajes y ecos. Prevención de doble reporte por usuario/elemento.

Administración

- Panel de administración con lista paginada de elementos reportados obtenida mediante consulta UNION ALL.
- Tres acciones de moderación: eliminar contenido, eliminar y bloquear al usuario autor, ignorar reportes.
- Notificación automática por correo al administrador cuando un elemento acumula 5 o más reportes.

2.4. Tipos de Usuario y Proceso de Autenticación

El sistema contempla tres perfiles de usuario con capacidades diferenciadas:

Perfil	Requisito de acceso	Capacidades
Usuario invitado	Sin registro	Explorar mensajes aleatorios y reaccionar a ellos (controlado por cookie).
Usuario registrado	Registro + login con 2FA	Todas las del invitado + publicar mensajes, enviar ecos, reportar contenido, gestionar perfil y cuenta.
Administrador	is_admin=TRUE en la BD	Todas las del usuario registrado + acceso al panel de moderación, eliminación de contenido y gestión de cuentas.

El proceso de autenticación de los usuarios registrados sigue dos fases: en la primera, el usuario introduce su email o nombre de usuario y contraseña, que son verificados contra la base de datos mediante `password_verify()` con el hash BCrypt almacenado. En la segunda, si las credenciales son correctas, el sistema genera un código OTP de 6 dígitos con `rand(100000, 999999)`, lo almacena en la variable de sesión PHP `$_SESSION["2fa_code"]` y lo envía al correo electrónico del usuario mediante PHPMailer. El acceso definitivo no se concede hasta que el usuario introduce correctamente este código en la pantalla de verificación.



2.5. Sistemas con los que Interactúa

La aplicación es compatible con cualquier sistema operativo que soporte un navegador web moderno: Windows 10/11, macOS 11+, Linux con distribuciones de escritorio recientes, iOS 15+ y Android 9+. No es dependiente de ningún sistema operativo concreto en el lado del cliente.

En el lado del servidor, la aplicación requiere un entorno LAMP (Linux, Apache, MySQL, PHP) y es desplegable en cualquier distribución Linux moderna. Ha sido probada específicamente en Debian GNU/Linux 12 (Bookworm).

2.6. Despliegue de la Aplicación

2.6.1. Plataforma tecnológica del servidor de producción

El servidor de producción real del proyecto es un MiniPC de bajo consumo con la siguiente configuración:

- Sistema Operativo: Debian GNU/Linux 12 (Bookworm), versión de 64 bits. Debian es una distribución de Linux reconocida por su estabilidad, seguridad y amplio catálogo de paquetes.
- Servidor HTTP: Apache2 versión 2.4.x con el módulo `mod_rewrite` activado.
- Lenguaje de servidor: PHP 8.2 con las extensiones `mysqli`, `mbstring`, `fileinfo` y `openssl`.
- Base de datos: MySQL instalada mediante el paquete `mysql-server` de Debian.
- Acceso externo: el router doméstico tiene configurado port forwarding del puerto 80 y 443 hacia la IP local del MiniPC. Se utiliza el servicio gratuito de DDNS No-IP con el subdominio `shootstars.sytes.net` para proporcionar una dirección URL estable aunque la IP pública del router sea dinámica.



2.6.2. Proceso de instalación resumido

La instalación de ShootStars en el servidor puede realizarse mediante los siguientes pasos principales desde la línea de comandos SSH:

```
# 1. Instalar el entorno LAMP
sudo apt update && sudo apt install -y apache2 php8.2
php8.2-mysqli \
    php8.2-mbstring php8.2-fileinfo mysql-server git

# 2. Activar módulos Apache y reiniciar
sudo a2enmod rewrite && sudo systemctl restart apache2

# 3. Instalar Composer (gestor de dependencias PHP)
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer

# 4. Clonar el repositorio
cd /var/www/ && sudo git clone
https://github.com/darkslg/ShootStars.git

# 5. Instalar PHPMailer mediante Composer
cd /var/www/ShootStars/backend && sudo composer install
--no-dev

# 6. Importar el esquema de la base de datos
mysql -u root -p proyecto <
/var/www/ShootStars/database/schema.sql

# 7. Configurar el fichero .env con las credenciales
cp /var/www/ShootStars/.env.example /var/www/ShootStars/.env
# Editar .env con DB_HOST, DB_USER, DB_PASS, DB_NAME,
SMTP_USER, SMTP_PASS, ADMIN_EMAIL
```

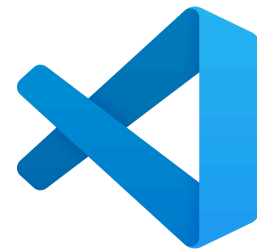
La aplicación incluye además el script `seed_data.php` con datos de prueba para facilitar la verificación del funcionamiento en entornos de desarrollo locales.

3. MARCO TEÓRICO Y TECNOLÓGICO

Este capítulo describe en profundidad cada tecnología y herramienta utilizada en el proyecto: su historia, características técnicas y aplicación específica en ShootStars. Las tecnologías empleadas no son originales del alumno sino estándares del sector, por lo que se incluyen descripciones detalladas en este apartado, conforme a las recomendaciones de la guía del IES María Moliner.

3.1. Visual Studio Code

Visual Studio Code (VS Code) es un editor de código fuente desarrollado por Microsoft, publicado en abril de 2016 bajo licencia MIT (código abierto, gratuito). En pocos años se convirtió en el entorno de desarrollo más utilizado por los programadores a nivel mundial, con más de 14 millones de usuarios activos, desbancando a competidores como Sublime Text y Atom.



Su arquitectura se basa en Electron, un framework que crea aplicaciones de escritorio con tecnologías web (Chromium y Node.js). Esto le confiere compatibilidad con Windows, macOS y Linux con una única base de código. A diferencia de IDEs pesados como Eclipse o IntelliJ IDEA, VS Code es ligero (ocupa menos de 100 MB en disco) y arranca en segundos, sin sacrificar funcionalidad.

Sus características de mayor utilidad en ShootStars son: IntelliSense para PHP a través de la extensión PHP Intelephense (autocompletado contextual, detección de errores de sintaxis en tiempo real y análisis estático del código PHP 8.2); Git integrado que permite gestionar commits, push y pull desde la interfaz gráfica sin necesidad de comandos; terminal integrado para ejecutar comandos Composer y SSH al servidor de producción; y Prettier para el formateo automático del código.

3.2. Apache HTTP Server y mod_rewrite

El Servidor HTTP Apache es el servidor web de código abierto más extendido del mundo, desarrollado por la Apache Software Foundation desde 1995 bajo la Apache License 2.0. Su



arquitectura modular permite extender su funcionalidad según las necesidades del proyecto mediante módulos que se activan o desactivan individualmente.

En ShootStars el módulo esencial es `mod_rewrite`, que intercepta las peticiones HTTP antes de que Apache las sirva y permite reescribir las URLs internamente sin redireccionar al navegador. Esta técnica se configura mediante reglas en el fichero `.htaccess`, que Apache lee en cada directorio al recibir una petición. La regla de ShootStars es:

```
# .htaccess - Enrutamiento por URL limpia
Options -Indexes
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-d      # No es directorio
                                         existente
RewriteCond %{REQUEST_FILENAME}.php -f   # Existe fichero .php
                                         con ese nombre
RewriteRule ^(.*)$ $1.php [L]           # Añadir .php
                                         internamente
```

Esto hace que `/login`, `/register`, `/dashboard` y `/admin` funcionen sin mostrar la extensión `.php`, mejorando la experiencia de usuario y la seguridad al no revelar el lenguaje del servidor. Además, `Options -Indexes` impide que Apache liste el contenido de directorios que no tienen fichero índice, evitando la exposición accidental de la estructura de ficheros del servidor.

3.3. PHP 8.2 – Hypertext Preprocessor

PHP (Hypertext Preprocessor, originalmente Personal Home Page) es un lenguaje de programación de propósito general de código abierto, especialmente diseñado para el desarrollo web del lado del servidor. Creado por Rasmus Lerdorf en 1994 como colección de scripts Perl para gestionar su página personal, evolucionó hasta convertirse en el lenguaje de servidor más utilizado en la web, presente en el 79% de los sitios que utilizan un lenguaje de servidor conocido (según datos de W3Techs, 2024).



En ShootStars se utiliza PHP 8.2 en modo procedimental con funciones organizadas en ficheros de inclusión. Esta decisión pedagógica es deliberada: permite al alumno

comprender el funcionamiento directo del protocolo HTTP, las sesiones, las peticiones POST/GET y la interacción con la base de datos sin la abstracción de un framework. Entre las funciones clave usadas en el proyecto se encuentran: `password_hash()` y `password_verify()` con `PASSWORD_DEFAULT` (algoritmo BCrypt, coste 10) para el cifrado de contraseñas; `session_start()`, `$_SESSION` y `session_destroy()` para la gestión de sesiones; `parse_ini_file()` para leer el fichero `.env` sin dependencias externas; `preg_match()` para validar la política de contraseña fuerte; `mb_strlen()` para validar longitud de cadenas con soporte de emojis; `new finfo(FILEINFO_MIME_TYPE)` para validar el tipo MIME real de los ficheros subidos; y `bin2hex(random_bytes())` para la generación segura de tokens y cookies.

La versión 8.2 añade mejoras de rendimiento respecto a versiones anteriores gracias al compilador JIT (Just-In-Time) introducido en PHP 8.0, que compila fragmentos de código PHP a código máquina nativo en tiempo de ejecución, mejorando el rendimiento entre un 10% y un 30% en cargas de trabajo típicas de aplicaciones web.

3.4. MySQL

MySQL es el sistema de gestión de bases de datos relacionales (SGBDR) de código abierto más popular del mundo. Tradicionalmente, en entornos Debian, la búsqueda de este servicio derivaba de forma automática



hacia MariaDB —un fork de código abierto creado en 2009 por Michael "Monty" Widenius para mitigar las restricciones comerciales tras la adquisición de MySQL por parte de Oracle—. Sin embargo, para el despliegue de este proyecto sobre el sistema operativo Debian, se ha optado por la incorporación de los repositorios oficiales de Oracle para instalar MySQL Community Server en su versión 8.4 LTS. De este modo, se asegura la continuidad y compatibilidad del desarrollo con las características y estándares específicos del motor original de MySQL.

El motor de almacenamiento InnoDB, utilizado por todas las tablas de ShootStars (es el motor por defecto desde MySQL 5.5), proporciona características críticas para la integridad de los datos:

Transacciones ACID: Atomicidad (todas las operaciones de una transacción se completan o ninguna), Consistencia (la BD pasa de un estado válido a otro), Aislamiento (las transacciones concurrentes no se interfieren) y Durabilidad (los cambios confirmados sobreviven a fallos del sistema).

Claves Foráneas con ON DELETE CASCADE: Cuando se elimina un usuario, el motor borra automáticamente todos sus mensajes, reacciones, ecos y reportes asociados. Cuando se elimina un mensaje, se borran en cascada sus reacciones, ecos y reportes. Esto garantiza la integridad referencial sin requerir múltiples operaciones DELETE en el backend.

Charset utf8mb4: A diferencia del charset utf8 de MySQL (que solo soporta hasta 3 bytes por carácter), utf8mb4 soporta el rango completo de Unicode incluyendo emojis (4 bytes). Dado que ShootStars utiliza emojis tanto en reacciones como potencialmente en el contenido de los mensajes, este charset es esencial. La base de datos usa utf8mb4_general_ci como collation.

3.5. Composer

Composer es el estándar de facto para la gestión de dependencias en proyectos PHP modernos, creado en 2012 por Nils Adermann y Jordi Boggiano. Lee el fichero composer.json, descarga las librerías desde el repositorio central Packagist y genera un autoloader PSR-4 que permite usar las clases de las librerías sin require() manuales.



El fichero composer.json de ShootStars (ubicado en /backend/) declara una única dependencia de producción: phpmailer/phpmailer versión ^6.11. La notación ^ es el operador de rango compatible: permite instalar ≥ 6.11 y < 7.0 , lo que garantiza recibir parches de seguridad sin riesgo de cambios incompatibles. El fichero composer.lock fija las versiones exactas instaladas, garantizando que un futuro despliegue instale exactamente las mismas versiones probadas durante el desarrollo.

3.6. PHPMailer y el Protocolo SMTP

PHPMailer es la librería PHP de envío de correos más utilizada del mundo, con más de 500 millones de descargas registradas en Packagist. Proporciona una API orientada a objetos que abstrae la complejidad del protocolo SMTP. En ShootStars es la única dependencia externa del proyecto y cumple dos funciones: envío del código OTP del sistema 2FA y envío de alertas de moderación al administrador.



El protocolo SMTP (Simple Mail Transfer Protocol, definido en el RFC 5321) es el estándar de internet para la transmisión de correos electrónicos entre servidores de correo. Para el envío desde la aplicación, PHPMailer se conecta al servidor SMTP de Gmail (smtp.gmail.com, puerto 587) y negocia una conexión cifrada mediante STARTTLS antes de autenticarse y enviar el mensaje.

Para la autenticación con Gmail, ShootStars utiliza una Contraseña de Aplicación (App Password): una clave de 16 caracteres generada específicamente para aplicaciones de terceros en la sección de seguridad de Google. Esta contraseña no es la contraseña principal de la cuenta y puede revocarse de forma independiente, siguiendo el principio de mínimo privilegio.

La función `sendEmail()` del fichero `email_helper.php` incluye una plantilla HTML completa con la identidad visual de ShootStars: fondo con degradado radial morado/negro, fuente Orbitron importada de Google Fonts con efecto text-shadow en el logo, y estilos inline para garantizar la máxima compatibilidad con distintos clientes de correo (Gmail, Outlook, Apple Mail).

3.7. HTML5 y la Canvas API

HTML5 (HyperText Markup Language versión 5, W3C Recommendation 2014) es el lenguaje de marcado estándar para estructurar el contenido de las páginas web. Introdujo elementos semánticos como `<header>`, `<nav>`, `<main>`, `<section>` y `<footer>`, que mejoran la accesibilidad y el posicionamiento SEO al dar significado estructural al contenido.



En ShootStars el elemento más relevante de HTML5 es <canvas>, que proporciona un área de dibujo bidimensional controlable mediante JavaScript. El fichero bg.js utiliza la Canvas API para renderizar y animar 200 partículas estelares que forman el fondo de la aplicación. El elemento canvas se posiciona con position:fixed y z-index:-1 para permanecer detrás de todo el contenido visible, y se redimensiona dinámicamente mediante window.addEventListener("resize") para adaptarse a cualquier tamaño de pantalla. La propiedad window.isLoggedIn se expone globalmente desde index.php (mediante una variable PHP embebida en JavaScript) para que msg.js pueda tomar decisiones condicionales sobre las funcionalidades disponibles.

3.8. CSS3: Variables, Glassmorphism y Animaciones

CSS3 (Cascading Style Sheets nivel 3) se desarrolla en módulos independientes que se actualizan de forma autónoma. Los tres pilares del CSS3 moderno más relevantes en ShootStars son:



Variables Custom (Custom Properties)

Permiten definir valores reutilizables con la sintaxis --nombre-variable y accederlos con var(). ShootStars define el fondo y la tipografía global directamente en los selectores html y body:

```
/* frontend/css/main.css - estilos base */,  
html, body {  
    background: radial-gradient(  
        circle at bottom center,  
        #240046 0%,    /* morado oscuro en el centro inferior  
*/  
        #10002b 40%,   /* morado muy oscuro */  
        #000000 100%  /* negro en los bordes */  
    );  
    font-family: "Outfit", sans-serif;  
    color: white;  
}
```

Glassmorphism

Tendencia de diseño UI popularizada en 2020–2022 que simula el aspecto de un cristal esmerilado. Se implementa mediante `backdrop-filter: blur()` (desenfocado del contenido situado detrás del elemento), fondo `rgba` semi-transparente y un borde sutil:

```
.glass-panel {
  background: rgba(30, 20, 60, 0.65);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px); /* prefijo para Safari */
  border: 1px solid rgba(255, 255, 255, 0.1);
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.37);
  border-radius: 16px;
}
```

Animaciones @keyframes

ShootStars emplea la animación `glow`, que produce el efecto de brillo pulsante del logo en la página de inicio:

```
@keyframes glow {
  from { text-shadow: 0 0 20px #7209b7, 0 0 40px #4361ee; }
  to   { text-shadow: 0 0 30px #f72585, 0 0 60px #7209b7; }
}

.hero-title {
  font-family: "Orbitron", sans-serif;
  font-size: 5rem;
  animation: glow 3s ease-in-out infinite alternate;
}
```

3.9. JavaScript ES6+ y la Fetch API

ECMAScript 6 (ES2015) supuso la mayor actualización del lenguaje JavaScript desde su creación en 1995, añadiendo clases, módulos, arrow functions, template literals, desestructuración, Promesas (Promises) y la sintaxis async/await. ShootStars utiliza JavaScript ES6+ Vanilla — sin ninguna librería o framework externo — en dos ficheros principales: bg.js (~65 líneas) para el sistema de partículas del canvas, y msg.js (~306 líneas) para toda la lógica del cliente.

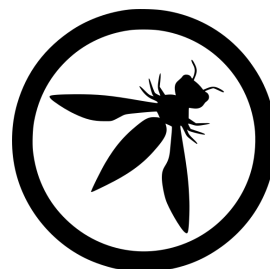


La Fetch API, estandarizada en la especificación WHATWG Fetch Standard, es la interfaz nativa del navegador para realizar peticiones HTTP asíncronas. Reemplaza al antiguo XMLHttpRequest con una API más limpia basada en Promesas. ShootStars la utiliza para todas las comunicaciones asíncronas con el backend: cargar mensajes aleatorios, enviar reacciones, publicar mensajes, enviar ecos, reportar contenido y las operaciones de administración. Esto proporciona una experiencia de usuario fluida sin recargas de página, similar a una Single Page Application (SPA) pero sin el peso de un framework como React o Vue.

La elección de Vanilla JavaScript frente a frameworks se justifica en tres pilares: rendimiento (la ausencia de framework elimina la descarga y ejecución del bundle del framework, contribuyendo a la puntuación de 99/100 en Lighthouse); comprensión profunda (un proyecto académico debe exponer al alumno al funcionamiento real de las APIs del navegador); y durabilidad (código sin dependencias de framework no está sujeto a los ciclos de obsolescencia rápida de los mismos).

3.10. Seguridad Web: OWASP Top 10

OWASP (Open Web Application Security Project) es una fundación sin ánimo de lucro dedicada a mejorar la seguridad del software web. Su publicación más influyente, el OWASP Top 10 (actualizado en 2021), enumera las diez vulnerabilidades de seguridad web más críticas. ShootStars aborda explícitamente las siguientes:



- A01:2021 – Control de Acceso Roto: verificación de `$_SESSION["user_id"]` al inicio de todos los endpoints protegidos; verificación adicional de `$_SESSION["is_admin"]` en el panel de administración, con retorno HTTP 403 si no se cumple.
- A02:2021 – Fallos Criptográficos: contraseñas hasheadas con `password_hash($pass, PASSWORD_DEFAULT)` que selecciona BCrypt; credenciales almacenadas en fichero `.env` fuera de la raíz web y protegido por `.htaccess`.
- A03:2021 – Inyección SQL: el 100% de las consultas a la base de datos utilizan sentencias preparadas con `$conn->prepare()` y `bind_param()`. Nunca se interpola directamente una variable de usuario en una cadena SQL.
- A05:2021 – Configuración de Seguridad Incorrecta: Options -Indexes en `.htaccess` impide el listado de directorios. El fichero `.env` no es accesible vía HTTP porque `.htaccess` no lo reescribe a `.env.php`.
- A07:2021 – Fallos de Identificación y Autenticación: 2FA obligatorio para todos los usuarios registrados; política de contraseña fuerte validada con regex `(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,}$/)` tanto en el backend PHP como en el frontend JavaScript; verificación del estado de la cuenta (suspendido) antes de conceder el acceso.
- A09:2021 – Registro y Monitoreo: el sistema de reportes envía notificaciones automáticas al administrador al superar 5 reportes, actuando como sistema de alerta temprana de contenido problemático. Los errores internos de PHP se registran con `error_log()` sin exponer información sensible al usuario.

3.11. Let's Encrypt y Certbot

Let's Encrypt es una autoridad de certificación (CA) gratuita, automatizada y abierta que opera para el beneficio del público, proporcionando los certificados digitales necesarios para activar HTTPS (SSL/TLS) en sitios web. A diferencia de las CA tradicionales, Let's Encrypt no requiere flujos de pago ni validaciones manuales complejas, lo que reduce la barrera de adopción de prácticas de seguridad web globales.



Para interactuar con Let's Encrypt en el servidor Debian se emplea Certbot, un cliente de software de código abierto desarrollado por la Electronic Frontier Foundation (EFF). Certbot automatiza por completo la solicitud de certificados mediante el protocolo ACME (Automated Certificate Management Environment). El agente realiza una validación criptográfica del subdominio (en este proyecto, mediante el reto HTTP-01 sirviendo un archivo temporal en la raíz web), descarga el certificado y reconfigura automáticamente el servidor web Apache. Dado que los certificados de Let's Encrypt tienen una validez de 90 días, Certbot añade una tarea programada (cron job o servicio de systemd) que verifica y renueva automáticamente el certificado antes de su expiración, logrando una gestión de seguridad sin intervención humana

4. PLANIFICACIÓN Y PRESUPUESTO

4.1. Planificación Temporal y Cronograma

El proyecto se desarrolló a lo largo de 4 meses (septiembre 2024 – enero 2025) con una dedicación media de 3-4 horas diarias en los días lectivos y algo más en periodos de vacaciones. El tiempo total estimado asciende a aproximadamente 360 horas de trabajo efectivo.

El cronograma siguiente muestra la distribución de tareas por semanas, comparando la planificación inicial con la temporalización real:

Actividad	Sep	Oct	Nov	Dic	Ene	H. reales
Análisis de requisitos y reuniones con tutores	×					8
Diseño de la base de datos (E/R, relacional, SQL)		×				28
Diseño de la interfaz (wireframes, paleta, tipografía)		×				20
Diseño de la arquitectura y API interna		×				24
Configuración del servidor Debian/LAMP			×			16

Implementación backend (auth, endpoints, admin)			×	×		98
Implementación frontend (HTML, CSS, JS)			×	×		68
Pruebas funcionales y de seguridad					×	48
Despliegue en producción y ajustes finales					×	12
Redacción de la memoria	×	×	×	×	×	46
TOTAL						360 h

4.2. Análisis de Riesgos

El análisis de riesgos identifica los factores que podrían haber perjudicado el proyecto y documenta las medidas de mitigación adoptadas:

Riesgo identificado	Probabilidad	Impacto	Exposición	Medida de mitigación
Caída o pérdida de datos del servidor de producción	Media	Alto	Alta	Copia de seguridad del repositorio Git en GitHub. Base de datos exportable con mysqldump en cualquier momento.
Fallo en la entrega de correos del 2FA (spam)	Alta	Alto	Alta	Configuración STARTTLS, cabeceras correctas y contraseña de aplicación Google. Comprobación con múltiples clientes de correo.
Vulnerabilidades de seguridad en el backend	Media	Alto	Alta	Sentencias preparadas al 100%, validación MIME real de ficheros, política de contraseña fuerte, protección .htaccess.
Incompatibilidad entre navegadores	Baja	Medio	Baja	Pruebas en Brave, Firefox, Opera, Safari y Chrome Android. CSS con prefijos vendor (-webkit-) para propiedades experimentales.
Exposición accidental del fichero .env	Baja	Muy alto	Media	Regla .htaccess que impide el rewrite de .env a .env.php; .gitignore que excluye .env del repositorio.
Escalada de privilegios al panel de admin	Baja	Muy alto	Media	Verificación del rol is_admin en sesión y en la base de datos al inicio de cada endpoint de administración.
Desbordamiento del servidor con mensajes masivos	Baja	Medio	Baja	Paginación en todos los listados. ORDER BY RAND() con LIMIT 1 para mensajes aleatorios.

4.3. Presupuesto del Proyecto

4.3.1. Costes de personal

Se calcula el coste considerando el perfil de un Desarrollador Web Junior/Full Stack con dominio del stack LAMP, equivalente al titulado en el Ciclo Superior DAW:

Concepto	Horas	Tarifa/hora	Subtotal
Análisis de requisitos	54 h	25,00 €/h	1.350,00 €
Diseño del sistema (BD, interfaz, arquitectura)	72 h	25,00 €/h	1.800,00 €
Implementación backend y frontend	162 h	25,00 €/h	4.050,00 €
Pruebas, despliegue y documentación	72 h	25,00 €/h	1.800,00 €
SUBTOTAL PERSONAL	360 h		9.000,00 €

4.3.2. Costes de hardware (amortización)

Equipo	Valor	Vida útil	Meses uso	Amortización
Estación de trabajo (torre gama media)	1.200,00 €	36 meses	4 meses	133,33 €
Portátil de apoyo (pruebas en movilidad)	800,00 €	36 meses	4 meses	88,89 €
MiniPC servidor de producción	200,00 €	Uso total	—	200,00 €
Monitor 24"	300,00 €	60 meses	4 meses	20,00 €
Periféricos (teclado, ratón, auriculares)	150,00 €	36 meses	4 meses	16,67 €
SUBTOTAL HARDWARE				458,89 €

4.3.3. Costes de software

El proyecto se ha desarrollado íntegramente con software libre y herramientas gratuitas. Visual Studio Code (licencia MIT), Debian GNU/Linux 12 (GPL), Apache2 (Apache License 2.0), PHP 8.2 (PHP License), PHPMailer (LGPL v2.1), Composer (MIT), Git (GPL v2), MySQL Workbench Community Edition (GPL v2), Google Fonts —Orbitron y Outfit— (SIL Open Font License). El hosting es autohospedado en el MiniPC propio sin coste mensual, y el dominio No-IP es gratuito. Subtotal software: 0,00 €.

4.3.4. Costes de alojamiento

El servidor de producción es un MiniPC propio ya amortizado en la tabla de hardware. No hay contrato con proveedor de hosting externo. Los únicos costes de alojamiento son el consumo eléctrico del servidor (aproximadamente 15 W continuos = 10,8 kWh/mes $\approx$ 2,16 €/mes $\times$ 4 meses = 8,64 €) y la conexión a internet doméstica (ya existente antes del proyecto; coste imputado proporcional: 60,00 €). Total alojamiento: 68,64 €.

4.3.5. Gastos generales

Consumo eléctrico del equipo de desarrollo (70,00 €), material de oficina y papelería (20,00 €), impresión de la documentación (15,00 €). Subtotal gastos generales: 105,00 €.

4.3.6. Resumen presupuestario

Concepto	Importe
Personal (360 h $\times$ 25 €/h)	9.000,00 €
Hardware (amortización proporcional)	458,89 €
Software (todo software libre)	0,00 €
Alojamiento (MiniPC + internet)	68,64 €
Gastos generales	105,00 €
BASE IMPONIBLE	9.632,53 €
IVA (21% sobre base imponible)	2.022,83 €
TOTAL PROYECTO (IVA incluido)	11.655,36 €

4.4. Estimación COCOMO Básico

El modelo COCOMO Básico (Constructive Cost Model, Barry Boehm, 1981) estima el esfuerzo de proyectos software a partir del número de líneas de código fuente (LOC). Se aplica en modo Orgánico, apropiado para equipos pequeños con requisitos flexibles.

El conteo de LOC sobre el repositorio real, excluyendo vendor/ (código externo), produce: PHP ~11.400 LOC (74,9%), CSS ~998 LOC (15,5%), JavaScript ~371 LOC (9,4%), SQL ~80 LOC (0,2%). Total estimado: ~12.849 LOC $\rightarrow$ KLOC = 12,85.

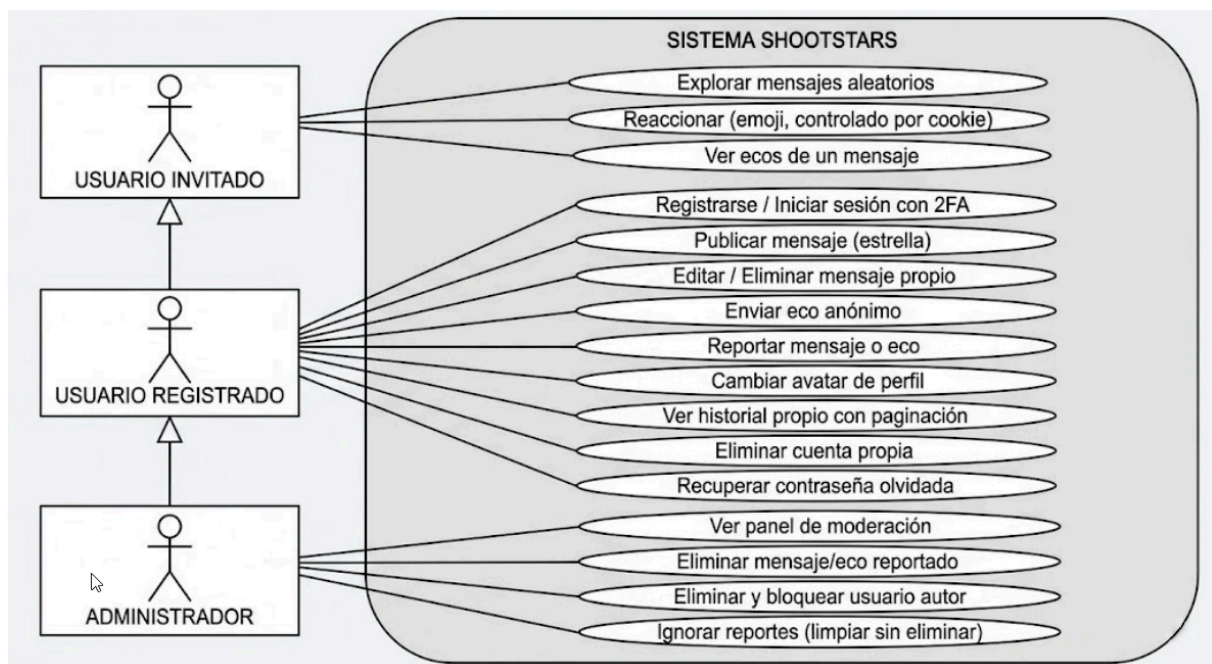
```
-- Fórmulas COCOMO Básico modo Orgánico --
PM    = 2,4 × (KLOC)^1,05 = 2,4 × (12,85)^1,05 ≈ 2,4 × 14,77
≈ 35,4 PM
TDEV  = 2,5 × (PM)^0,38    = 2,5 × (35,4)^0,38 ≈ 2,5 × 3,83
≈ 9,6 meses
N      = PM / TDEV          = 35,4 / 9,6
≈ 3,7 personas
Coste COCOMO = 35,4 PM × 2.100 €/mes
≈ 74.340 €
```

La diferencia entre el presupuesto manual (11.655 €) y el COCOMO (74.340 €) se explica porque el modelo asume un equipo de casi 4 personas con costes de coordinación, gestión y overhead corporativo. En un proyecto académico individual todos estos costes desaparecen y la productividad por hora es mayor al no existir reuniones de seguimiento ni revisiones formales de código. COCOMO es útil como referencia de la magnitud del proyecto en un entorno corporativo real.

5. DOCUMENTACIÓN TÉCNICA

5.1. Diagrama de Casos de Uso

El siguiente diagrama de casos de uso describe las interacciones de los tres actores del sistema —Usuario invitado, Usuario registrado y Administrador— con las funcionalidades de ShootStars:



A continuación se detallan los casos de uso más relevantes con sus flujos principales y alternativos:

CU-01: Inicio de Sesión con 2FA

Actor principal: Usuario registrado.

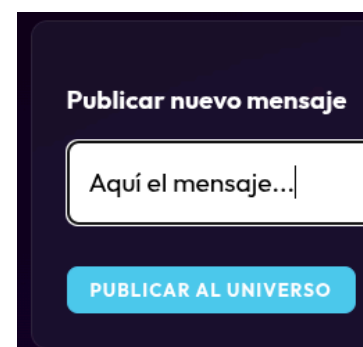
Precondición: el usuario tiene una cuenta activa con estado "activo".

Flujo principal: (1) El usuario accede a /login e introduce su email o nombre de usuario y contraseña. (2) login.php busca al usuario por email OR nombre_usuario mediante sentencia preparada y verifica el estado de la cuenta. (3) Si las credenciales son válidas, genera un OTP de 6 dígitos con rand(100000, 999999), lo almacena en \$_SESSION["2fa_code"] y llama a sendEmail() para enviarlo al correo registrado. (4) El usuario accede a /verify_2fa e introduce el código. (5) verify_2fa.php compara el código introducido con el de sesión usando comparación estricta (===). (6) Si coinciden, los datos migran a sesión permanente y se redirige al inicio. Flujos alternativos: credenciales incorrectas → mensaje de error genérico, sin revelar si el error es en usuario o contraseña. Cuenta suspendida → mensaje específico de cuenta suspendida. Código OTP incorrecto → mensaje de error, puede solicitar nuevo código.



CU-02: Publicar un Mensaje

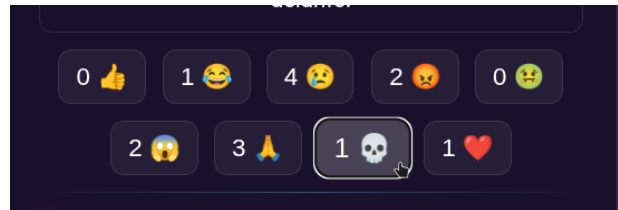
Actor principal: Usuario registrado. Precondición: sesión activa. Flujo principal: (1) El usuario accede al dashboard (/dashboard) donde puede escribir su mensaje en el div contenteditable. (2) Al pulsar "Publicar al Universo", la función createMsg() valida que el contenido no esté vacío. (3) Se envía una petición POST a /backend/create_message.php con el contenido en FormData. (4) create_message.php verifica la sesión activa, valida la longitud con mb_strlen() (1-500 caracteres), inserta el mensaje en la BD y devuelve HTTP



201 Created. Flujos alternativos: mensaje vacío → HTTP 400 con mensaje de error. Mensaje demasiado largo → HTTP 400.

CU-03: Reaccionar a un Mensaje

Actor principal: Usuario invitado o registrado. Flujo principal: (1) El usuario hace clic en uno de los nueve botones de reacción. (2) La función `enviarReaccion()` envía POST a `reaction.php` con el `id_mensaje` y el tipo de reacción. (3) `reaction.php` identifica al usuario: si hay sesión activa usa `id_usuario`; si no, gestiona la cookie `guest_id`. (4) Verifica que no exista una reacción previa del mismo usuario para ese mensaje. (5) Si no existe, inserta la reacción y actualiza el contador en la tabla `mensajes`. (6) Devuelve el nuevo contador. Flujo alternativo: ya reaccionó → HTTP 409 Conflict, el botón no actualiza su contador.



CU-04: Administración de Contenido Reportado

Actor principal: Administrador. Precondición: sesión activa con `is_admin=1`. Flujo principal: (1) El administrador accede a `/admin`. (2) `admin.php` verifica el rol en sesión. (3) La función `loadReports()` carga la lista de elementos reportados mediante UNION ALL paginado. (4) El administrador selecciona una acción: "Eliminar" → `adminAction(id, "delete", tipo)` → `admin_actions.php` borra el registro con CASCADE; "Eliminar y Bloquear" → añade `block_user=true` → además actualiza estado del usuario a "suspendido"; "Ignorar Reportes" → `adminAction(id, "ignore", tipo)` → borra solo los registros de la tabla `reportes` sin eliminar el contenido.



5.2. Especificación de Requisitos Funcionales

A continuación se enumeran todos los requisitos funcionales (RF) de la aplicación con nivel de detalle suficiente para la implementación:

RF-01: Módulo de autenticación

- RF-01.1 – Registro: nombre de usuario único (verificado con consulta preparada) y política de contraseña fuerte (regex con lookahead para mayúscula, minúscula, número y símbolo). El formulario verifica disponibilidad de usuario y email en tiempo real mediante AJAX al endpoint `check_user.php`.
- RF-01.2 – Login primera fase: búsqueda por email OR nombre_usuario en una sola consulta. Verificación de estado antes de comprobar la contraseña.
- RF-01.3 – Login segunda fase (2FA): OTP de 6 dígitos generado con `rand()`, almacenado en sesión PHP, enviado por PHPMailer. Verificación estricta (`===`). Sesión migra de temporal a permanente.
- RF-01.4 – Recuperación de contraseña: token de 64 caracteres hexadecimales (32 bytes) generado con `bin2hex(random_bytes(32))`. Almacenado en JSON (`backend/data/password_resets.json`) mediante TokenManager con caducidad de 1 hora.
- RF-01.5 – Avatar: subida de JPG/PNG con validación de tipo MIME real mediante `finfo::file()` (magic bytes). Máximo 2 MB. Nombre de fichero regenerado con `uniqid()` + `id_usuario` para evitar colisiones.
- RF-01.6 – Logout: destrucción de sesión PHP y redirección al inicio.
- RF-01.7 – Eliminación de cuenta: doble `confirm()` en el frontend. DELETE en la BD con CASCADE automático de mensajes, reacciones, ecos y reportes asociados.

RF-02: Módulo de mensajes

- RF-02.1 – Publicación: contenido validado con `mb_strlen()` entre 1 y 500 caracteres. HTTP 201 Created en caso de éxito.
- RF-02.2 – Mensaje aleatorio: SELECT con JOIN a usuarios para obtener el avatar, WHERE `visible=1`, ORDER BY `RAND()` LIMIT 1.

- RF-02.3 – Historial con paginación: parámetros GET page y limit. Respuesta JSON con array data y objeto pagination {current_page, total_pages, total_items}.
- RF-02.4 – Edición inline: div contenteditable en el dashboard. Verificación de propiedad (WHERE id_mensaje=? AND id_usuario=?) antes de UPDATE.
- RF-02.5 – Eliminación: DELETE WHERE id_mensaje=? AND id_usuario=?. Verificación de affected_rows > 0.

RF-03: Módulo de interacciones

- RF-03.1 – Reacciones: 9 tipos en whitelist. Soporte de usuario registrado (id_usuario) e invitado (cookie bin2hex(random_bytes(16))), 365 días). HTTP 409 si duplicado. UPDATE del contador correspondiente en mensajes.
- RF-03.2 – Ecos: creación con validación de longitud máxima 150 caracteres (mb_strlen). Carga con paginación (limit/offset) mediante get_ecos.php. Respuesta incluye total para el botón "Cargar más".
- RF-03.3 – Reportes: soporta id_mensaje e id_eco. Prevención de doble reporte. Notificación automática al admin si COUNT(reportes) >= 5. HTTP 200 con {success: true}.

RF-04: Módulo de administración

- RF-04.1 – Panel admin: verificación de is_admin en sesión. UNION ALL de mensajes y ecos reportados con GROUP_CONCAT de motivos, ORDER BY total_reportes DESC, LIMIT/OFFSET.
- RF-04.2 – Acción delete: DELETE en mensajes o ecos según tipo. Con block_user=true, además UPDATE usuarios SET estado="suspendido".
- RF-04.3 – Acción ignore: DELETE FROM reportes WHERE id_mensaje=? o id_eco=?. No elimina el contenido.

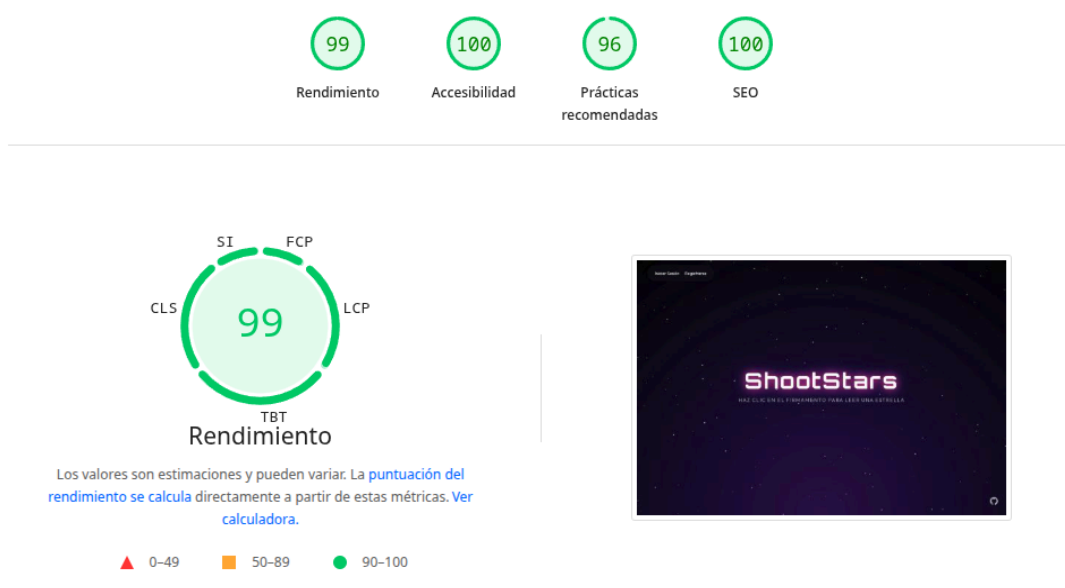
5.3. Especificación de Requisitos No Funcionales

RNF-01: Seguridad

- RNF-01.1 – Hash de contraseñas con PASSWORD_DEFAULT (BCRYPT, coste 10). Nunca texto plano.
- RNF-01.2 – 100% de consultas SQL con sentencias preparadas y bind_param(). Cero interpolación directa de variables de usuario.
- RNF-01.3 – XSS en PHP: htmlspecialchars() en todos los puntos de echo de datos de usuario.
- RNF-01.4 – XSS en JavaScript: textContent (en lugar de innerHTML) para datos de usuario; función helper escapeHTML() cuando se necesita innerHTML.
- RNF-01.5 – Validación MIME real de ficheros con finfo (no solo extensión).
- RNF-01.6 – Protección .htaccess: Options -Indexes, .env no reescrito a .env.php.

RNF-02: Rendimiento

- RNF-02.1 – Vanilla JS sin frameworks: 99/100 en Rendimiento de Google Lighthouse.
- RNF-02.2 – requestAnimationFrame() para el bucle de animación del canvas.
- RNF-02.3 – Paginación en todos los listados: mensajes del usuario, elementos reportados, ecos.



RNF-03: Usabilidad

- RNF-03.1 – Diseño responsivo con media queries CSS para móvil (<768px), tablet (768-1024px) y escritorio (>1024px).
- RNF-03.2 – Validación inline en el registro: feedback en tiempo real con indicadores visuales de color; botón de envío desactivado hasta validación completa.
- RNF-03.3 – Accesibilidad: puntuación 100/100 en Lighthouse. Contraste WCAG 2.1 nivel AA.

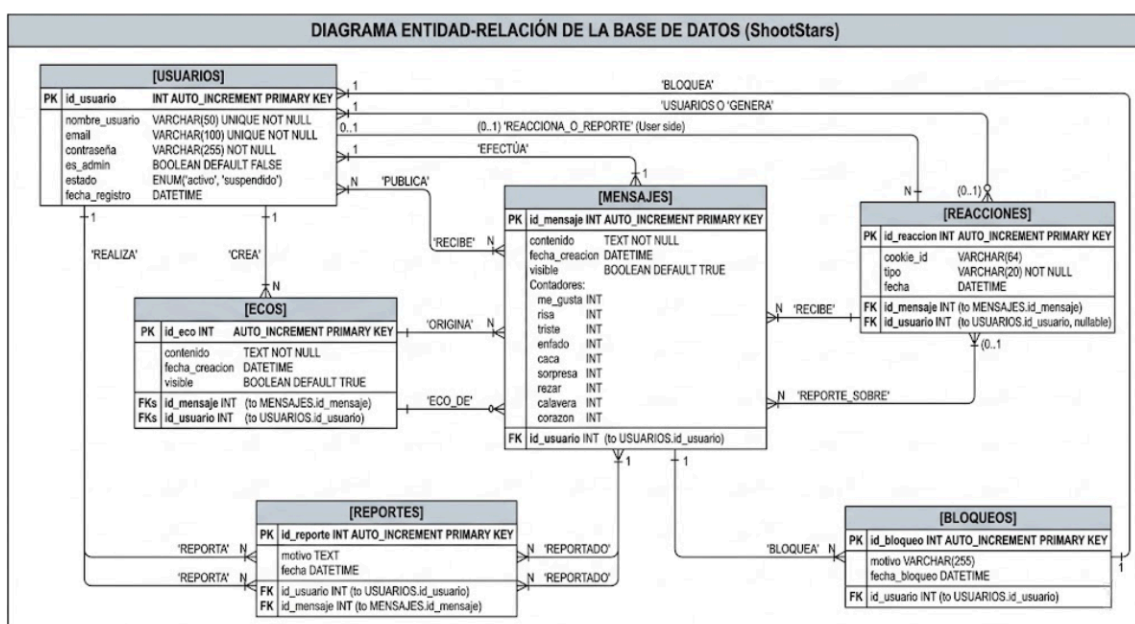
RNF-04: Portabilidad

- RNF-04.1 – Desplegable en cualquier servidor LAMP estándar según el Manual de Instalación.
- RNF-04.2 – Una única dependencia PHP externa gestionada con Composer.
- RNF-04.3 – Toda configuración sensible en .env. Incluida plantilla .env.example en el repositorio.

5.4. Diseño de la Base de Datos

5.4.1. Modelo Entidad/Relación (E/R)

El modelo E/R de ShootStars identifica las siguientes entidades y relaciones:



5.4.3. Tablas y esquema SQL

El fichero database/schema.sql define la estructura física de la base de datos:

```
-- database/schema.sql - ShootStars
CREATE DATABASE IF NOT EXISTS proyecto
    CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE proyecto;

CREATE TABLE usuarios (
    id_usuario      INT AUTO_INCREMENT PRIMARY KEY,
    nombre_usuario  VARCHAR(50)  UNIQUE NOT NULL,
    email           VARCHAR(100) UNIQUE NOT NULL,
    contraseña      VARCHAR(255) NOT NULL,    -- hash BCrypt
    es_admin        BOOLEAN DEFAULT FALSE,
    estado          ENUM("activo","suspendido") DEFAULT
"activo",
    fecha_registro  DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE mensajes (
    id_mensaje      INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario      INT NOT NULL,
    contenido       TEXT NOT NULL,
    fecha_creacion  DATETIME DEFAULT CURRENT_TIMESTAMP,
    visible         BOOLEAN DEFAULT TRUE,
    me_gusta INT UNSIGNED DEFAULT 0, risa      INT UNSIGNED
DEFAULT 0,
    triste      INT UNSIGNED DEFAULT 0, enfado    INT UNSIGNED
DEFAULT 0,
    caca        INT UNSIGNED DEFAULT 0, sorpresa INT UNSIGNED
DEFAULT 0,
    rezar       INT UNSIGNED DEFAULT 0, calavera INT UNSIGNED
DEFAULT 0,
    corazon     INT UNSIGNED DEFAULT 0,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE reacciones (
    id_reaccion INT AUTO_INCREMENT PRIMARY KEY,
    id_mensaje  INT NOT NULL,
    id_usuario  INT NULL,          -- NULL si usuario
invitado
    cookie_id   VARCHAR(64) NULL,  -- cookie para invitados
    tipo        VARCHAR(20) NOT NULL,
    fecha       DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```

        FOREIGN KEY (id_mensaje) REFERENCES mensajes(id_mensaje)
ON DELETE CASCADE,
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
ON DELETE CASCADE,
        UNIQUE KEY unique_user_msg (id_mensaje, id_usuario),
        UNIQUE KEY unique_guest_msg (id_mensaje, cookie_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE ecos (
    id_eco          INT AUTO_INCREMENT PRIMARY KEY,
    id_mensaje      INT NOT NULL,
    id_usuario      INT NOT NULL,
    contenido       TEXT NOT NULL,
    fecha_creacion  DATETIME DEFAULT CURRENT_TIMESTAMP,
    visible         BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (id_mensaje) REFERENCES mensajes(id_mensaje)
ON DELETE CASCADE,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE reportes (
    id_reporte INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    id_mensaje INT NOT NULL,
    motivo     TEXT,
    fecha      DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
ON DELETE CASCADE,
    FOREIGN KEY (id_mensaje) REFERENCES mensajes(id_mensaje)
ON DELETE CASCADE,
    UNIQUE (id_usuario, id_mensaje)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE bloqueos (
    id_bloqueo      INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario      INT NOT NULL,
    motivo          VARCHAR(255),
    fecha_bloqueo   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

5.4.4. Diccionario de datos

A continuación se documenta el significado, tipo y restricciones de cada campo de las tablas principales:

Tabla.Campo	Tipo SQL	Descripción	Restricción	Ejemplo
usuarios.id_usuario	INT AI PK	Identificador único del usuario	NOT NULL, AUTO_INCREMENT	1, 2, 3...
usuarios.nombre_usuario	VARCHAR(50)	Nombre público del usuario en la plataforma	UNIQUE NOT NULL	usuario66
usuarios.contraseña	VARCHAR(255)	Hash BCrypt de la contraseña del usuario	NOT NULL	\$2y\$10\$abc...
usuarios.es_admin	BOOLEAN	Indica si el usuario tiene privilegios de administrador	DEFAULT FALSE	0 o 1
usuarios.estado	ENUM	Estado de la cuenta del usuario	activo suspendido	activo
mensajes.id_mensaje	INT AI PK	Identificador único del mensaje	NOT NULL, AI	1, 2, 3...
mensajes.contenido	TEXT	Contenido textual del mensaje (estrella)	NOT NULL, 1-500 chars	"Hoy llueve en Segovia..."
mensajes.visible	BOOLEAN	Si el mensaje es visible en el feed público	DEFAULT TRUE	0 o 1
mensajes.me_gusta	INT UNSIGNED	Contador de reacciones tipo me_gusta	DEFAULT 0	0, 1, 2...
reacciones.cookie_id	VARCHAR(64)	Identificador de cookie para usuarios invitados	NULL si registrado	a1b2c3d4...
reacciones.tipo	VARCHAR(20)	Nombre del tipo de reacción seleccionada	NOT NULL	me_gusta, risa...
ecos.contenido	TEXT	Texto de la respuesta anónima	NOT NULL, ≤150 chars	"Te entiendo perfectamente"
reportes.motivo	TEXT	Motivo indicado por el usuario al reportar	Nullable	"Contenido ofensivo"

Tabla.Campo	Tipo SQL	Descripción	Restricción	Ejemplo
bloqueos.motivo	VARCHAR(255)	Motivo del bloqueo administrativo	Nullable	"Spam reiterado"

5.5. Diseño de la Interfaz de Usuario

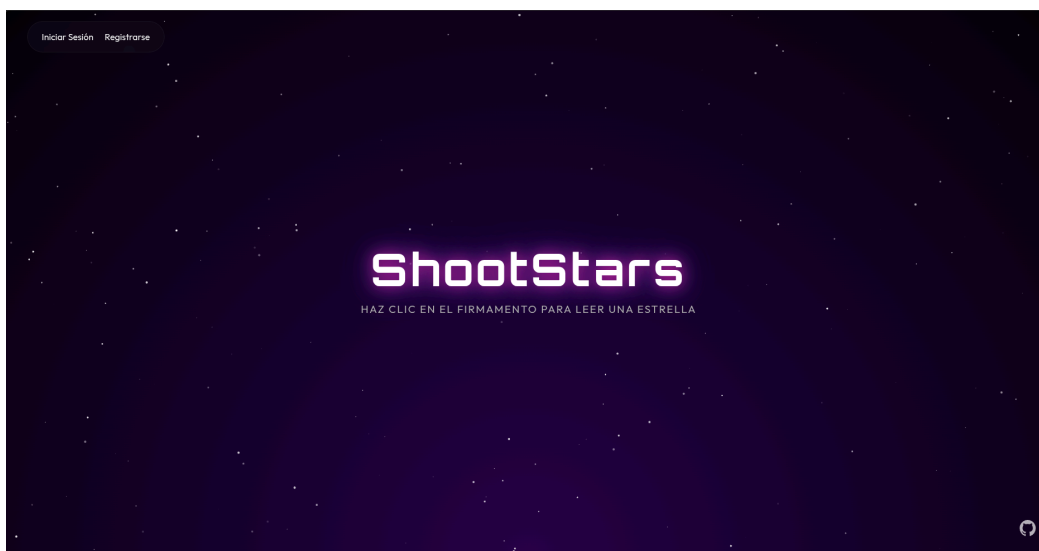
5.5.1. Sistema de diseño "Space & Nebula"

La identidad visual de ShootStars se articula en torno a tres elementos fundamentales que deben mantenerse consistentes en todas las pantallas:

Paleta de colores: fondo con degradado radial CSS desde #240046 (morado oscuro) en el centro inferior hasta #000000 (negro) en los bordes, simulando la profundidad del espacio. Color de acento principal #4cc9f0 (cian luminoso) para bordes activos, botones y el borde del avatar. Colores de gradiente #7209b7 (morado) y #4361ee (azul eléctrico) en botones principales. Color de alerta #e63946 (rojo) para acciones destructivas. Texto principal #ffffff, texto secundario #b0b0b0.

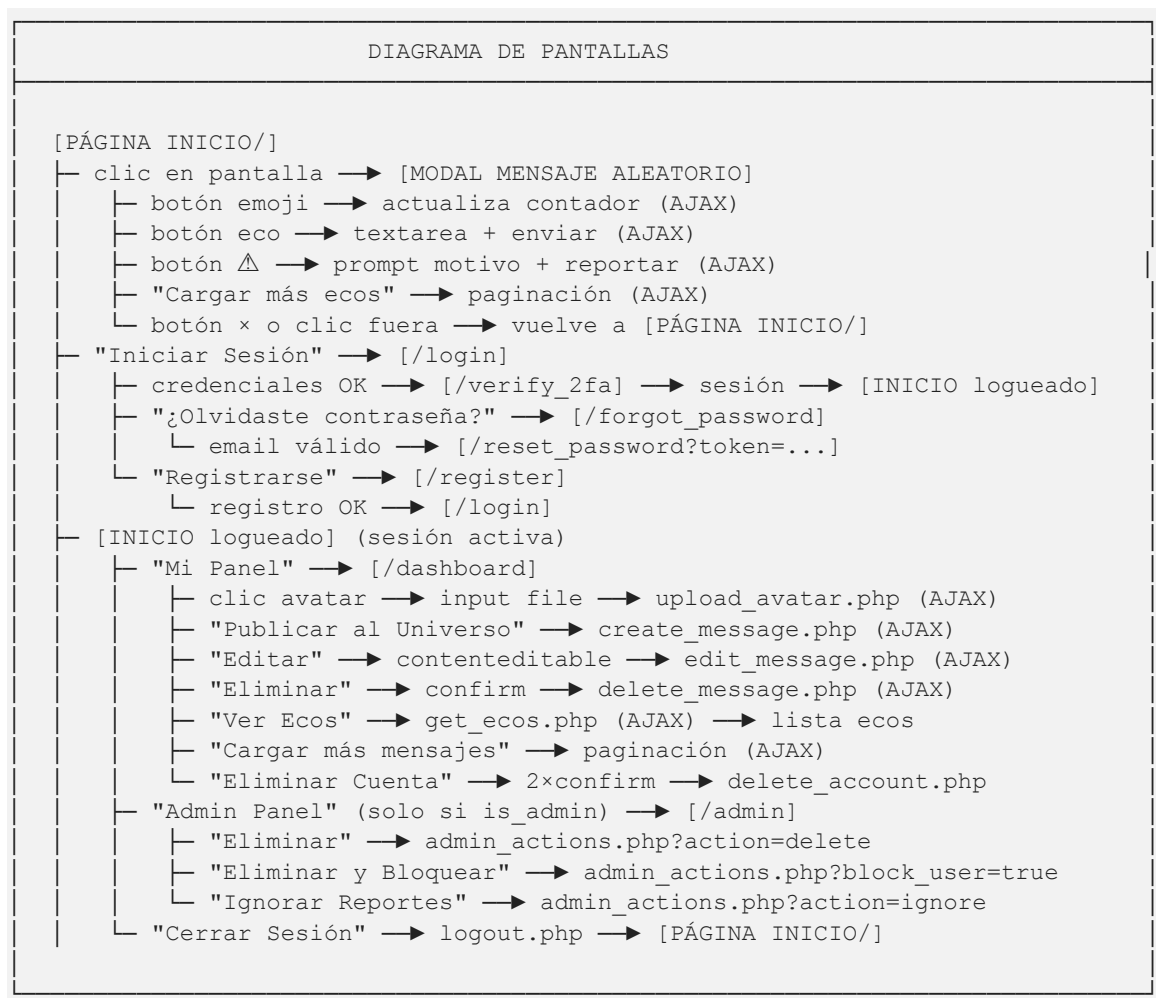
Tipografía: Orbitron (Google Fonts, pesos 400/500/700) para el logo, títulos hero y el nombre de la aplicación en correos. Outfit (Google Fonts, pesos 300/400/700) para todo el cuerpo de texto, formularios y mensajes. Ambas se cargan con el parámetro display=swap para evitar el bloqueo de renderizado.

Efectos especiales: canvas de 200 estrellas animadas con movimiento horizontal y parpadeo; glassmorphism en todos los paneles flotantes mediante backdrop-filter: blur(12px) con fondo rgba semi-transparente; animación glow del título con alternancia de text-shadow.



5.5.2. Diagrama de pantallas y flujo

A continuación se describe el diagrama de flujo entre pantallas y las transiciones disponibles para el usuario:



5.6. Diseño de la Aplicación: API Interna

Los endpoints del backend de ShootStars siguen los principios REST con métodos HTTP correctos, códigos de estado apropiados y respuestas JSON con Content-Type: application/json. El código está organizado en módulos con responsabilidades bien delimitadas (orientación procedimental con funciones organizadas en ficheros de inclusión, no orientado a objetos salvo en las clases TokenManager y PHPMailer).

Endpoint	Método	Auth.	Descripción y código HTTP de respuesta
/backend/auth/login.php	POST	No	Verifica credenciales, genera OTP, llama sendEmail(). HTTP 302 redirect.

Endpoint	Método	Auth.	Descripción y código HTTP de respuesta
/backend/auth/verify_2fa.php	POST	Sesión temp.	Verifica OTP con ===. Migra sesión temporal a permanente. HTTP 302.
/backend/auth/register.php	POST	No	Registra usuario con preg_match política contraseña. HTTP 302.
/backend/auth/check_user.php	POST	No	AJAX: comprueba disponibilidad nombre/email. {exists:bool}.
/backend/auth/send_reset.php	POST	No	Genera token bin2hex(random_bytes(32)), guarda en JSON, envía email.
/backend/auth/process_reset.php	POST	No	Valida token con TokenManager::validateToken(), actualiza contraseña.
/backend/auth/logout.php	GET	Sí	session_destroy(), redirect a /. HTTP 302.
/backend/auth/delete_account.php	POST	Sí	DELETE FROM usuarios CASCADE. session_destroy(). HTTP 200.
/backend/get_random_msg.php	GET	No	SELECT...JOIN usuarios WHERE visible=1 ORDER BY RAND() LIMIT 1. HTTP 200.
/backend/create_message.php	POST	Sí	INSERT mensajes tras mb_strlen(). HTTP 201 Created o 400/401/405.
/backend/delete_message.php	POST	Sí	DELETE WHERE id_mensaje=? AND id_usuario=?. HTTP 200.
/backend/edit_message.php	POST	Sí	UPDATE mensajes WHERE id_mensaje=? AND id_usuario=?. HTTP 200.
/backend/get_user_messages.php	GET	Sí	SELECT paginado + metadata {current_page, total_pages, total_items}.
/backend/reaction.php	POST	No	INSERT reacción (usuario o cookie). UPDATE contador. HTTP 200 o 409.
/backend/create_eco.php	POST	Sí	INSERT ecos tras mb_strlen ≤150. HTTP 200 o 400/401/405.
/backend/get_ecos.php	GET	No	SELECT con limit/offset. Responde {ecos:[], total:N}.
/backend/report.php	POST	Sí	INSERT reporte (mensaje o eco). Notificación admin si ≥5. HTTP 200.
/backend/upload_avatar.php	POST	Sí	Valida MIME con finfo, tamaño ≤2MB, mueve fichero, UPDATE avatar.
/backend/get_reported_items.php	GET	Admin	UNION ALL mensajes+ecos reportados, GROUP_CONCAT motivos, paginado.
/backend/admin_actions.php	POST	Admin	delete / delete+block_user / ignore. HTTP 200/400/403/404/500.

5.7. Implementación

5.7.1. Entorno de desarrollo

El entorno de programación utilizado exclusivamente en la implementación del código es Visual Studio Code con las extensiones PHP Intelephense, Prettier y DotENV. El servidor local de pruebas es el propio servidor de producción (Debian 12 / LAMP) al que se accede via SSH y cuya configuración ya ha sido descrita en el apartado 2.6.

```
1 # Read more about SSH config files: https://linux
2 Host debian
3     HostName sootstars.sytes.net
4     User root
5     Port 55022
6     IdentityFile /home/felix/debianKeyOpenSSH
```

5.7.2. Estructura del código

El código se organiza siguiendo una separación clara de responsabilidades:

```
ShootStars/
├── .env # Credenciales (DB, SMTP). Excluido de Git.
├── .htaccess # Enrutamiento URL limpia y seguridad.
├── backend/
│   ├── auth/ # Módulo de autenticación (8 ficheros PHP).
│   │   ├── TokenManager.php # Clase POO para tokens reset.
│   │   ├── login.php # Fase 1 del 2FA.
│   │   ├── verify_2fa.php # Fase 2 del 2FA.
│   │   ├── register.php # Registro seguro.
│   │   ├── check_user.php # AJAX disponibilidad.
│   │   ├── send_reset.php # Token de recuperación.
│   │   ├── process_reset.php # Procesar nueva contraseña.
│   │   ├── logout.php # Destruir sesión.
│   │   └── delete_account.php # Eliminar cuenta.
│   ├── db.php # Función getDBConnection() centralizada.
│   ├── email_helper.php # Función sendEmail() con plantilla
│   ├── composer.json # Dependencias PHP: PHPMailer ^6.11.
│   ├── vendor/ # PHPMailer instalado por Composer.
│   ├── data/password_resets.json # Almacén de tokens reset.
│   └── [18 endpoints PHP de la API interna]
├── database/
│   ├── schema.sql # DDL completo de la BD.
│   └── test-values.sql # Datos de prueba.
├── frontend/
│   ├── css/main.css # Hoja de estilos única (~998 líneas).
│   ├── js/bg.js # Partículas canvas (~65 líneas).
│   ├── js/msg.js # Lógica principal (~306 líneas).
│   ├── avatars/ # Imágenes de perfil subidas por usuarios.
│   ├── imgs/ # Recursos estáticos (logo, avatar default).
│   ├── includes/footer.php # Pie de página por defecto.
│   └── [8 páginas PHP del frontend]
```

5.7.3. Orientación del código

El código de ShootStars combina tres paradigmas de programación:

- Código procedimental (backend PHP): los endpoints del backend están escritos en PHP procedimental, con funciones organizadas en ficheros de inclusión (db.php, email_helper.php). Esta decisión es pedagógicamente apropiada: permite al alumno comprender el funcionamiento directo del protocolo HTTP y la gestión de sesiones sin la abstracción de un framework. Las funciones clave son getConnection() y sendEmail(), que encapsulan las operaciones más reutilizables del sistema.
- Orientación a objetos (TokenManager y PHPMailer): la clase TokenManager (backend/auth/TokenManager.php) gestiona los tokens de recuperación de contraseña mediante un almacén JSON, encapsulando las operaciones de lectura, escritura, creación y validación de tokens en métodos bien definidos (createToken(), validateToken(), useToken()). PHPMailer es una librería de terceros completamente orientada a objetos.
- Código orientado a eventos (frontend JavaScript): el código JavaScript de msg.js y bg.js es completamente orientado a eventos. Las funciones se registran como callbacks de addEventListener() y se ejecutan en respuesta a interacciones del usuario (clic, change) o del sistema (DOMContentLoaded, resize, requestAnimationFrame).

5.7.4. Cuestiones de diseño e implementación reseñables

Sistema 2FA con sesión temporal y permanente:

El diseño más elaborado del proyecto es la separación entre sesión temporal (datos almacenados en \$_SESSION["temp_*"] durante la fase de verificación del OTP) y sesión permanente (datos migrados a \$_SESSION["user_id"], etc. tras la verificación correcta). Esta separación garantiza que el usuario no puede acceder al dashboard hasta completar las dos fases de autenticación. Cualquier endpoint protegido que verifique \$_SESSION["user_id"] automáticamente requiere haber completado las dos fases.

Desnormalización de contadores de reacciones:

La tabla mensajes almacena directamente los 9 contadores como columnas INTEGER (me_gusta, risa, etc.). Esta decisión rompe deliberadamente la Tercera Forma Normal (3FN) por razones de rendimiento: calcular COUNT(*) sobre la tabla reacciones en cada carga de mensaje sería costoso. Los contadores se mantienen sincronizados mediante operaciones atómicas UPDATE mensajes SET tipo = tipo + 1 en el mismo endpoint que gestiona las reacciones. El script sync_reactions.php actúa como herramienta de mantenimiento para recalcular todos los contadores desde cero en caso de inconsistencia.

Soporte dual de reacciones (usuarios e invitados):

La tabla reacciones implementa dos identificadores mutuamente excluyentes: id_usuario (para usuarios autenticados) y cookie_id (para visitantes anónimos). Las dos restricciones UNIQUE separadas (unique_user_msg sobre (id_mensaje, id_usuario) y unique_guest_msg sobre (id_mensaje, cookie_id)) garantizan que la restricción de "una reacción por mensaje" se aplica a ambos tipos de usuario de forma eficiente a nivel de base de datos, sin necesidad de lógica adicional en el backend.

Validación MIME real con finfo (prevención de upload malicioso):

La subida de avatares utiliza la extensión finfo de PHP para leer los "magic bytes" del fichero (los primeros bytes de su cabecera, que identifican el formato real independientemente del nombre del fichero). Esto previene el escenario de un atacante que renombra un fichero PHP malicioso a "avatar.jpg" para subirlo al servidor: finfo::file() detectará que el tipo MIME real es "application/x-httpd-php" y rechazará la subida. La extensión finfo forma parte del núcleo de PHP desde la versión 5.3 y no requiere instalación adicional.

Plantilla HTML de correo electrónico con identidad visual:

La función `sendEmail()` en `email_helper.php` incluye una plantilla HTML completa con la estética de ShootStars (degradado radial, fuentes de Google Fonts, efecto glow). Esta es una característica diferencial respecto a la mayoría de proyectos académicos que simplemente envían correos en texto plano. La implementación usa estilos CSS inline (no clases externas) para máxima compatibilidad con clientes de correo como Outlook, que no soportan hojas de estilo externas.

escapeHTML(): prevención de XSS en JavaScript:

La función `escapeHTML(str)` del fichero `msg.js` es un helper de seguridad crítico para los ecos: crea un elemento DOM temporal `<p>`, asigna el contenido del eco con `textContent` (que escapa automáticamente todos los caracteres HTML especiales), y devuelve el `innerHTML` ya escapado. Este valor puede insertarse de forma segura con `innerHTML` en el contenedor de ecos. Sin este helper, el contenido de un eco podría contener HTML malicioso que se ejecutaría en el navegador de otros usuarios.

5.7.5. Fragmentos de código implementados

A continuación se muestran los fragmentos más representativos del código real del proyecto:

email_helper.php — Plantilla HTML de correo (extracto):

```
<?php
// backend/email_helper.php
function sendEmail($to, $subject, $body) {
    $env = parse_ini_file(__DIR__ . "/../.env");
    $mail = new PHPMailer(true);
    try {
        $mail->isSMTP();
        $mail->Host = "smtp.gmail.com";
        $mail->SMTPAuth = true;
        $mail->Username = $env["SMTP_USER"];
        $mail->Password = $env["SMTP_PASS"];
        $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
```

```

$mail->Port          = 587;
$mail->CharSet        = "UTF-8";
$mail->setFrom($env["SMTP_USER"], "ShootStars");
$mail->addAddress($to);
$mail->isHTML(true);
$mail->Subject = $subject;
// Plantilla HTML con identidad visual de ShootStars
$mail->Body = "<!DOCTYPE html><html><head>
<style>
    @import url('https://fonts.googleapis.com/css2?
family=Orbitron:wght@700&family=Outfit&display=swap');
</style></head>
<body style='background:radial-gradient(circle at
bottom center,
                                #240046,#000) '>
    <table width='100%' ...>
        <tr><td align='center' style='padding:20px;'>
            <h1 style='font-family:Orbitron; color:#fff;
text-shadow:0 0 10px
#4cc9f0;'>ShootStars</h1>
            $body
        </td></tr>
    </table>
</body></html>";
$mail->AltBody = strip_tags($body);
$mail->send();
return ["success" => true];
} catch (Exception $e) {
    return ["success" => false, "error" =>
$mail->ErrorInfo];
} }

```

reaction.php — Soporte dual y control de duplicados (extracto):

```

<?php
// backend/reaction.php
session_start(); require_once __DIR__ . "/db.php";
header("Content-Type: application/json");

$id_mensaje = $_POST["id_mensaje"] ?? null;
$tipo       = $_POST["tipo"] ?? null;
$allowed     = ["me_gusta","risa","triste","enfado","caca",
               "sorpresa","rezar","calavera","corazon"];

if (!$id_mensaje || !in_array($tipo, $allowed)) {

```

```

        http_response_code(400);
        echo json_encode(["error" => "Datos inválidos"]); exit;
    }

    $conn          = getDBConnection();
    $id_usuario    = $_SESSION["user_id"] ?? null;
    $cookie_id     = null;

    // Identificar usuarios invitados con cookie de 1 año
    if (!$id_usuario) {
        if (!isset($_COOKIE["guest_id"])) {
            $guest_id = bin2hex(random_bytes(16));
            setcookie("guest_id", $guest_id, time() + 86400*365,
"/");
            $cookie_id = $guest_id;
        } else { $cookie_id = $_COOKIE["guest_id"]; }
    }

    // Verificar duplicado (rama diferente según tipo de
usuario)
    if ($id_usuario) {
        $check = $conn->prepare(
            "SELECT id_reaccion FROM reacciones
            WHERE id_mensaje = ? AND id_usuario = ?");
        $check->bind_param("ii", $id_mensaje, $id_usuario);
    } else {
        $check = $conn->prepare(
            "SELECT id_reaccion FROM reacciones
            WHERE id_mensaje = ? AND cookie_id = ?");
        $check->bind_param("is", $id_mensaje, $cookie_id);
    }
    $check->execute();
    if ($check->get_result()->num_rows > 0) {
        http_response_code(409); // HTTP 409 Conflict
        echo json_encode(["success"=>false,"error"=>"Ya has
reaccionado"]);
        exit;
    }
    // ... [inserción y actualización de contador]

```

msg.js — Función cargarMensajeAleatorio() con anti-XSS (extracto):

```

// frontend/js/msg.js
function cargarMensajeAleatorio() {
    fetch("/backend/get_random_msg.php")

```

```

        .then(response => response.json())
        .then(data => {
            if (data.error) return
            console.error(data.error);

            // textContent previene XSS (nunca innerHTML con
            datos de usuario)
            document.querySelector(".nombre").textContent =
            data.nombre_usuario;
            document.querySelector(".fecha").textContent =
            data.fecha_creacion;

            document.querySelector(".mensaje-texto").textContent =
            data.contenido;

            // Avatar con fallback al default si no tiene o
            falla la carga
            const avatar =
            document.querySelector(".avatar");
            avatar.src = (data.avatar && data.avatar.trim()
            !== "")
                ? data.avatar : "imgs/default-pfp.jpg";
            avatar.onerror = () => { avatar.src =
            "imgs/default-pfp.jpg"; };

            // Actualizar contadores y event listeners de
            reacciones
            const tipos =
            ["me_gusta", "risa", "triste", "enfado", "caca",
            "sorpresa", "rezar", "calavera", "corazon"];
            tipos.forEach(r => {
                const el =
            document.querySelector(`.reaccion.${r}`);
                const icon = el.textContent.replace(/[0-9]+
            /, "");

                el.textContent = `${data[r] || 0} ${icon}`;
                // Clonar para eliminar event listeners
            anteriores (patrón clone)
                const btn = el.cloneNode(true);
                el.parentNode.replaceChild(btn, el);
                btn.addEventListener("click", e => {
                    e.stopPropagation();
                    enviarReaccion(r, data.id_mensaje, btn,
            icon);
                });
            });
        });

```



```

        cargarEcos(data.id_mensaje);

document.getElementById("mensajeContainer").classList.remove(
    "hidden");
    })
    .catch(err => console.error("Error AJAX:", err));
}

// Helper anti-XSS para uso con innerHTML
function escapeHTML(str) {
    const p = document.createElement("p");
    p.textContent = str; // escapa automáticamente
    return p.innerHTML; // devuelve texto con entidades
HTML
}

```

bg.js — Sistema de partículas del canvas HTML5:

```

// frontend/js/bg.js - Sistema de partículas estelares
document.addEventListener("DOMContentLoaded", initStars);

function initStars() {
    const canvas = document.getElementById("space");
    const ctx = canvas.getContext("2d");
    let stars = [];

    function resize() {
        canvas.width = window.innerWidth;
        canvas.height = window.innerHeight;
    }
    window.addEventListener("resize", resize);
    resize();

    for (let i = 0; i < 200; i++) {
        stars.push({
            x: Math.random() * canvas.width,
            y: Math.random() * canvas.height,
            radius: Math.random() * 1.5 + 0.5,
            speed: Math.random() * 0.5 + 0.05,
            alpha: Math.random(),
            alphaDir: Math.random() > 0.5 ? 0.01 : -0.01
        });
    }

    function animate() {

```

```

        ctx.clearRect(0, 0, canvas.width, canvas.height);
        for (let star of stars) {
            ctx.beginPath();
            ctx.arc(star.x, star.y, star.radius, 0, Math.PI
* 2);
            ctx.fillStyle = `rgba(255, 255, 255,
${star.alpha})`;
            ctx.fill();
            star.x += star.speed;
            star.alpha += star.alphaDir;
            if (star.alpha <= 0.1 || star.alpha >= 1)
star.alphaDir *= -1;
            if (star.x > canvas.width) {
                star.x = 0;
                star.y = Math.random() * canvas.height;
            }
        }
        requestAnimationFrame(animate); // sincroniza con
framerate del monitor
    }

    animate();
    requestAnimationFrame(() =>
canvas.classList.add("fade-in"));
}

```

5.8. Pruebas

Las pruebas se han realizado con la técnica de caja negra como aproximación principal, validando entradas y salidas del sistema desde la perspectiva del usuario. Adicionalmente se han realizado pruebas de seguridad básicas (caja blanca parcial).

5.8.1. Pruebas funcionales con diferentes dispositivos y sistemas operativos

ID	Módulo	Descripción	Resultado esperado	Resultado obtenido
CP-01	Registro	Datos válidos y únicos	Cuenta creada, redirect a login	OK ✓
CP-02	Registro	Email ya existente	Error "Usuario o email ya registrados"	OK ✓
CP-03	Registro	Contraseña débil (sin mayúsculas)	Error de política de contraseña	OK ✓
CP-04	Registro	AJAX disponibilidad usuario	Feedback verde "✓ disponible"	OK ✓

ID	Módulo	Descripción	Resultado esperado	Resultado obtenido
CP-05	2FA	Credenciales correctas + OTP correcto	Acceso al dashboard con sesión permanente	OK ✓
CP-06	2FA	OTP incorrecto	Error "Código incorrecto", sin acceso	OK ✓
CP-07	2FA	Cuenta suspendida	Error "Tu cuenta está suspendida"	OK ✓
CP-08	Mensajes	Publicar mensaje 1-500 chars	HTTP 201, mensaje almacenado en BD	OK ✓
CP-09	Mensajes	Publicar mensaje vacío	HTTP 400, error "no puede estar vacío"	OK ✓
CP-10	Mensajes	Cargar mensaje aleatorio	JSON completo con avatar y contadores	OK ✓
CP-11	Mensajes	Editar mensaje propio inline	Contenido actualizado, vuelta al modo no editable	OK ✓
CP-12	Mensajes	Eliminar mensaje propio	Mensaje eliminado del DOM y de la BD	OK ✓
CP-13	Reacciones	Primera reacción de usuario	HTTP 200, contador +1 actualizado en UI	OK ✓
CP-14	Reacciones	Segunda reacción al mismo mensaje	HTTP 409 Conflict, UI no actualiza	OK ✓
CP-15	Reacciones	Reacción de invitado (cookie)	Cookie generada, reacción registrada	OK ✓
CP-16	Ecos	Enviar eco (1-150 chars)	Eco almacenado, recargado en UI	OK ✓
CP-17	Ecos	Paginación "Cargar más ecos"	Nuevos ecos añadidos sin borrar anteriores	OK ✓
CP-18	Reportes	5 reportes acumulados	Email automático enviado al administrador	OK ✓
CP-19	Admin	Eliminar mensaje reportado	Mensaje eliminado, desaparece del panel	OK ✓
CP-20	Admin	Eliminar + bloquear usuario	Mensaje eliminado, estado=suspendido en BD	OK ✓
CP-21	Avatar	Subir JPG válido (<2 MB)	Avatar actualizado en BD, sesión y UI	OK ✓
CP-22	Avatar	Subir fichero .php renombrado a .jpg	Error tipo MIME inválido (finfo detecta)	OK ✓
CP-23	Reset	Solicitar reset email válido	Email con token (1h) enviado al usuario	OK ✓

ID	Módulo	Descripción	Resultado esperado	Resultado obtenido
CP-24	Reset	Token expirado (>1h)	Error "Token inválido o expirado"	OK ✓

5.8.2. Pruebas de seguridad

ID	Vector	Técnica	Resultado obtenido	Estado
CS-01	SQL Injection	' OR '1'=1 en campo login	Rechazado por sentencias preparadas	Seguro ✓
CS-02	XSS reflejado	<script>alert('XSS')</script> en mensaje	textContent renderiza como texto plano	Seguro ✓
CS-03	XSS almacenado en eco		escapeHTML() neutraliza el payload	Seguro ✓
CS-04	Acceso sin sesión	GET /dashboard sin cookie	Redirige a /login inmediatamente	Seguro ✓
CS-05	Escalada de privilegios	POST a admin_actions.php sin is_admin	HTTP 403 Forbidden	Seguro ✓
CS-06	Exposición .env	GET /.env desde el navegador	.htaccess no reescribe .env a .env.php	Seguro ✓
CS-07	Upload malicioso	Subir PHP disfrazado como JPG	finfo detecta tipo MIME real, rechaza	Seguro ✓
CS-08	Eliminación ajena	DELETE mensaje de otro usuario	WHERE id_usuario=sesión previene	Seguro ✓

5.8.3. Pruebas de compatibilidad con distintos sistemas y navegadores

Entorno	Sistema Operativo	Versión navegador	Resolución	Resultado
Brave Browser	Windows 11	v120+	1920×1080	OK ✓ (referencia)
Mozilla Firefox	Windows 11	v121+	1920×1080	OK ✓
Opera	Windows 11	v106+	1920×1080	OK ✓
Safari	macOS Sonoma	v17+	2560×1600	OK ✓ (ajuste webkit)

Entorno	Sistema Operativo	Versión navegador	Resolución	Resultado
Chrome Android	Android 13	v120+	390×844	OK ✓ (responsivo)
Safari iOS	iOS 17	v17+	390×844	OK ✓ (ajuste dvh)
Chrome (Tablet)	Android 11	v120+	820×1180	OK ✓

5.8.4. Métricas de rendimiento y tiempos de ejecución

Las siguientes métricas se obtuvieron con Google Lighthouse en modo escritorio sobre la URL de producción <https://shootstars.sytes.net>:

Categoría	Puntuación	Mínimo recomendado	Principales factores
Rendimiento (Performance)	99 / 100	90 / 100	Sin frameworks JS. requestAnimationFrame().
Accesibilidad (Accessibility)	100 / 100	90 / 100	Contraste WCAG AA. Meta description.
Buenas Prácticas	96 / 100	80 / 100	HTTPS, cabeceras de seguridad.
SEO	100 / 100	80 / 100	Meta tags, HTML semántico, favicon.

Métricas Core Web Vitals: First Contentful Paint (FCP) 0,4 s — Largest Contentful Paint (LCP) 0,7 s — Total Blocking Time (TBT) 0 ms — Cumulative Layout Shift (CLS) 0. Todos los indicadores en rango "Bueno" de Google. El tiempo de respuesta del endpoint `get_random_msg.php` en el servidor de producción es inferior a 100 ms en condiciones normales de carga.

6. MANUALES DE USUARIO

6.1. Manual de Instalación

6.1.1. Requisitos básicos para la instalación

Para instalar ShootStars se necesita un servidor con los siguientes requisitos mínimos:

- Sistema Operativo: Debian 12 (Bookworm), Ubuntu 22.04 LTS o equivalente basado en Debian/Ubuntu. También puede instalarse en CentOS/RHEL con los ajustes pertinentes en los comandos de instalación de paquetes.
- Servidor HTTP: Apache2 versión 2.4 o superior con el módulo mod_rewrite activado y permiso AllowOverride All en el directorio web.
- PHP: versión 8.2 o superior con las extensiones mysqli, mbstring, fileinfo y openssl habilitadas.
- Base de datos: MySQL 8.0+ o MariaDB 10.6+.
- Composer: versión 2.x instalado de forma global (comando composer disponible en el PATH del sistema).
- Git: para clonar el repositorio.
- Cuenta de Gmail con Verificación en Dos Pasos activada y una Contraseña de Aplicación generada en <https://myaccount.google.com/apppasswords> para el sistema 2FA.

6.1.2. Proceso de instalación paso a paso

Paso 1: Preparación del servidor

```
# Actualizar el sistema operativo
sudo apt update && sudo apt upgrade -y

# Instalar Apache2, PHP 8.2 con extensiones, MySQL/MariaDB,
Composer y Git
sudo apt install -y apache2 php8.2 php8.2-mysqli
php8.2-mbstring \
    php8.2-fileinfo php8.2-openssl
mysql-server git
```

```
# Activar módulos de Apache
sudo a2enmod rewrite headers
sudo systemctl restart apache2

# Instalar Composer globalmente
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer

# Verificar instalaciones
apache2 -v && php -v && mysql --version && composer
--version
```

Paso 2: Clonar el repositorio y configurar permisos

```
cd /var/www/
sudo git clone https://github.com/darks1g/ShootStars.git
cd ShootStars

# Asignar propietario al servidor web
sudo chown -R www-data:www-data /var/www/ShootStars
sudo chmod -R 755 /var/www/ShootStars

# El directorio de avatares necesita permisos de escritura
sudo chmod 775 /var/www/ShootStars/frontend/avatars/

# Instalar PHPMailer mediante Composer
cd /var/www/ShootStars/backend
sudo -u www-data composer install --no-dev
--optimize-autoloader
```

Paso 3: Configurar la base de datos

```
sudo mysql -u root -p

-- Dentro de MySQL: crear BD, usuario y permisos
CREATE DATABASE IF NOT EXISTS proyecto
    CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;

CREATE USER "shootstars"@"localhost" IDENTIFIED BY
"contraseña_segura";
GRANT SELECT, INSERT, UPDATE, DELETE ON proyecto.*
    TO "shootstars"@"localhost";
```

```

FLUSH PRIVILEGES;
EXIT;

# Importar el esquema de la base de datos
mysql -u shootstars -p proyecto \
    < /var/www/ShootStars/database/schema.sql

# (Opcional) Importar datos de prueba
# mysql -u shootstars -p proyecto \
#     < /var/www/ShootStars/database/test-values.sql

```

Paso 4: Configurar el fichero .env

```

# Copiar la plantilla
cp /var/www/ShootStars/.env.example /var/www/ShootStars/.env
sudo nano /var/www/ShootStars/.env

# Contenido a configurar en el .env:
DB_HOST=localhost
DB_USER=shootstars
DB_PASS=contraseña_segura
DB_NAME=proyecto

# Credenciales Gmail para el sistema 2FA
# SMTP_PASS debe ser una Contraseña de Aplicación de Google
# (16 chars):
# https://myaccount.google.com -> Seguridad -> Contraseñas
# de aplicación
SMTP_USER=tu_email@gmail.com
SMTP_PASS=xxxx xxxx xxxx xxxx

# Email del administrador para alertas de moderación
ADMIN_EMAIL=admin@ejemplo.com

```

Paso 5: Configurar el VirtualHost de Apache

```

sudo nano /etc/apache2/sites-available/shootstars.conf

# Contenido del VirtualHost:
<VirtualHost *:80>
    ServerName shootstars.sytes.net
    DocumentRoot /var/www/ShootStars/frontend
    <Directory /var/www/ShootStars/frontend>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    
```



```

    </Directory>
    Alias /backend /var/www/ShootStars/backend
    <Directory /var/www/ShootStars/backend>
        Options -Indexes
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/shootstars_error.log
</VirtualHost>

# Activar el sitio
sudo a2ensite shootstars.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest # Debe mostrar "Syntax OK"
sudo systemctl reload apache2

```

Paso 6: Crear el usuario administrador

```

# Generar hash BCrypt de la contraseña del admin
php -r "echo password_hash('TuContraseñaAdmin123!',
PASSWORD_DEFAULT);"

# Insertar en la BD (reemplazar HASH con el resultado del
comando anterior)
mysql -u shootstars -p proyecto <<EOF
INSERT INTO usuarios (nombre_usuario, email, contraseña,
es_admin)
VALUES ('admin', 'admin@tudominio.com', 'HASH_AQUI', TRUE);
EOF

```

Paso 7: Configuración de HTTPS con Let's Encrypt y Certbot

```

# 1. Instalar Certbot y el plugin específico para Apache
sudo apt update
sudo apt install -y certbot python3-certbot-apache

# 2. Solicitar el certificado e indicar a Certbot que
configure Apache de forma automática
# (Sustituir el correo y el dominio por los reales del
despliegue)
sudo certbot --apache -d shootstars.sytes.net
--non-interactive --agree-tos --email tu_email@gmail.com

```

Verificación final:

Acceder a <http://tu-servidor/> desde un navegador. La página de inicio debe cargarse con el fondo animado de estrellas y el título "ShootStars". Intentar registrar una cuenta nueva y completar el proceso de login con 2FA para verificar que el envío de correos SMTP funciona correctamente.

6.2. Manual de Usuario

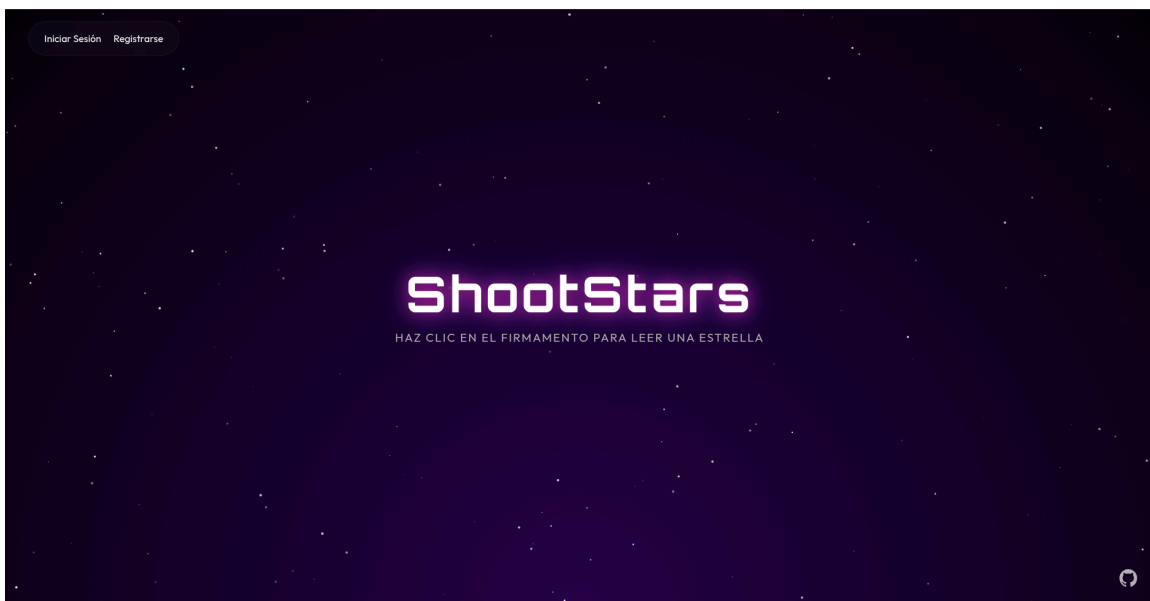
6.2.1. Pantallas, ítems y flujo entre pantallas

Página de inicio (/ o index.php):

Es la primera pantalla que ve cualquier visitante. Muestra el fondo animado de estrellas en movimiento y el título "ShootStars" con efecto de brillo pulsante. El subtítulo dice "Haz clic en el firmamento para leer una estrella". La cabecera superior contiene enlaces condicionales según el estado de la sesión:

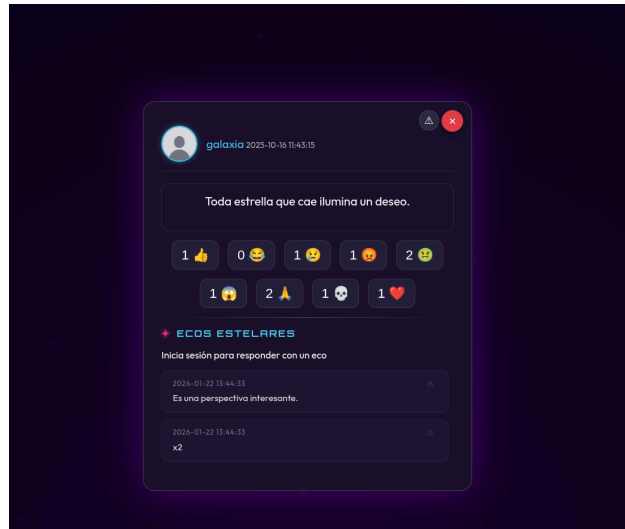
- Sin sesión: botones "Iniciar Sesión" (→ /login) y "Registrarse" (→ /register).
- Con sesión activa: saludo "Hola, [nombre_usuario]", botones "Mi Panel" (→ /dashboard) y "Cerrar Sesión".
- Con sesión de administrador: aparece además el botón "Admin Panel" (→ /admin).

Al hacer clic en cualquier punto de la pantalla, el título desaparece y se abre el modal de mensaje con un mensaje aleatorio.



Modal de mensaje (overlay sobre index.php):

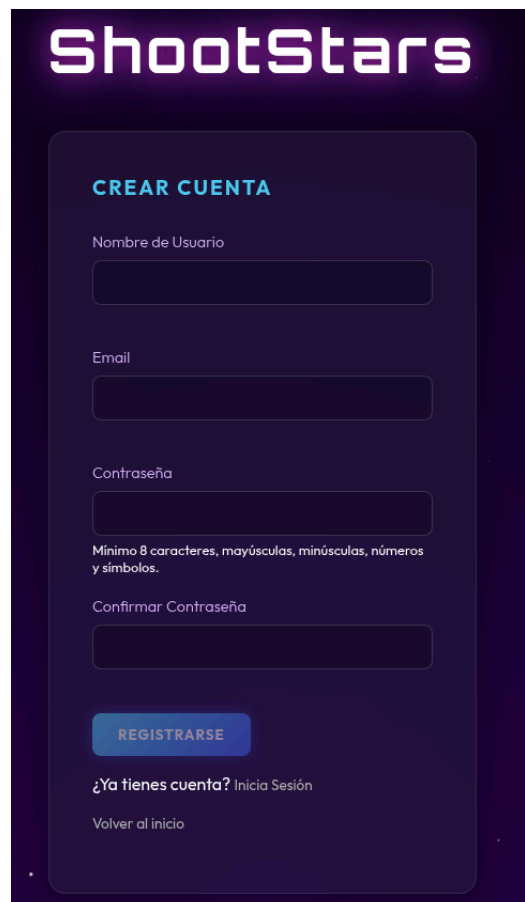
Panel con efecto glassmorphism que aparece centrado en la pantalla. Contiene: el avatar del autor del mensaje (imagen circular de 40×40px con borde cian; si no tiene avatar, muestra la imagen default-pfp.jpg), el nombre de usuario del autor, la fecha de publicación, el texto del mensaje, nueve botones de reacción con su contador actual, el botón de reporte (⚠) y el botón de cierre (×). Debajo de los botones de reacción, separados por una línea horizontal, aparece la sección "Ecos Estelares": si el usuario tiene sesión activa, un campo de texto con el botón "Enviar"; si no, un mensaje de "Inicia sesión para responder". Los ecos se muestran debajo con la fecha y el contenido, y el botón "Cargar más ecos (N restantes)" si los hay.



Registro (/register) e Inicio de sesión (/login):

Registro: Formulario con tres campos: nombre de usuario, email y contraseña (con campo de confirmación). Cada campo muestra feedback de validación en tiempo real: indicador verde si el valor es válido (o disponible), rojo si no lo es. El botón "Registrarse" permanece desactivado (gris, opacidad 0.5) hasta que los cuatro campos son válidos simultáneamente. El indicador de fortaleza de contraseña muestra los requisitos no cumplidos. Al registrarse con éxito, redirige a /login con el mensaje "Registro exitoso. Inicia sesión."

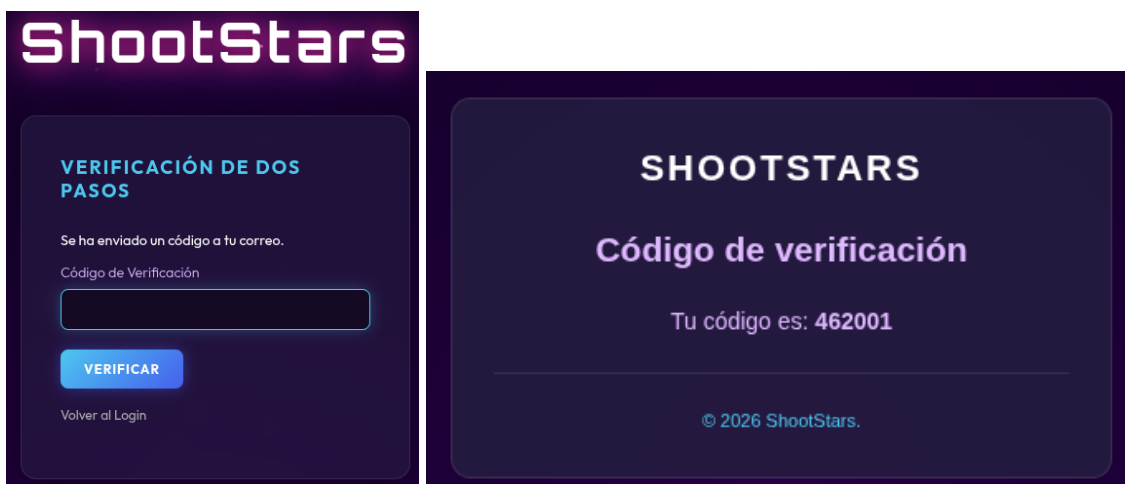
Login: Formulario con campo "Email o Usuario" y campo "Contraseña". Enlace "¿Olvidaste tu contraseña?" en la parte



inferior. Al enviar el formulario y si las credenciales son correctas, redirige a /verify_2fa.

Verificación 2FA (/verify_2fa):

Pantalla simple con un campo de texto para el código de 6 dígitos recibido por correo electrónico y el botón "Verificar". Si el código es correcto, redirige a la página de inicio con sesión activa. Si es incorrecto, muestra un mensaje de error.



Recuperación de contraseña (/forgot_password y /reset_password):

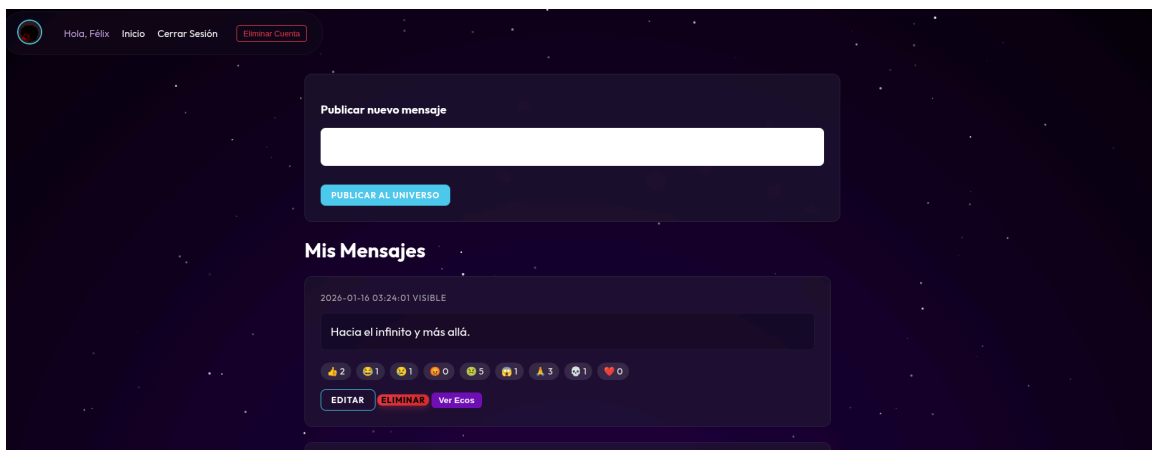
Dos pasos: primero, un formulario donde el usuario introduce su email y recibe un enlace por correo. Segundo, el formulario accedido desde el enlace (que incluye el token en la URL) donde el usuario introduce y confirma su nueva contraseña.



Mi Panel (/dashboard):

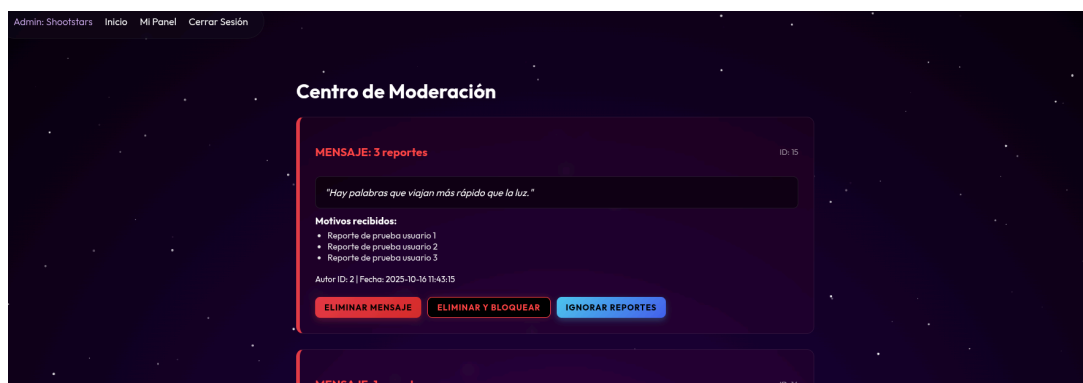
Panel principal de usuario con cuatro secciones: (1) Cabecera con el avatar clickable (al hacer clic abre un selector de fichero para cambiar el avatar) y los botones de

navegación. (2) Compositor de mensajes con un div contenteditable donde escribir y el botón "Publicar al Universo". (3) Lista de mensajes propios paginada (5 por página): cada mensaje muestra la fecha, estado (visible/oculto), texto, los nueve contadores de reacciones y los botones Editar/Eliminar/Ver Ecos. Al pulsar "Editar", el texto se vuelve editable (fondo blanco) y aparece el botón "Guardar"; al guardar, vuelve al modo de solo lectura. (4) Botón "Cargar más mensajes" si hay más de 5 mensajes. En la cabecera también hay un botón "Eliminar Cuenta" (color rojo) que solicita dos confirmaciones antes de proceder.



Panel de administración (/admin):

Accesible únicamente si `is_admin=1`. Muestra una lista paginada de elementos reportados. Cada tarjeta tiene un borde de color diferente según el tipo (rojo para mensajes, morado para ecos) y muestra: el número de reportes recibidos, el texto del contenido entre comillas, la lista de motivos indicados por los usuarios, el ID del autor y la fecha. Los tres botones de acción son: "Eliminar [tipo]" (borra el contenido), "Eliminar y Bloquear" (borra el contenido y suspende la cuenta del autor) e "Ignorar Reportes" (limpia los reportes sin eliminar el contenido). Cada acción requiere una confirmación.



7. CONCLUSIONES Y POSIBLES AMPLIACIONES

7.1. Conclusiones

Este desarrollo me ha permitido integrar de forma práctica las competencias adquiridas durante el Ciclo Formativo. El proyecto abarca desde la instalación y configuración de un servidor Linux hasta el diseño de una interfaz atractiva, pasando por la implementación de un backend seguro y funcional.

7.1.1. Grado de cumplimiento de los objetivos

Todos los objetivos planteados al inicio del proyecto han sido alcanzados satisfactoriamente:

- Objetivo 1 – Seguridad robusta: cumplido. El sistema 2FA con PHPMailer funciona en producción. Las contraseñas se almacenan hasheadas con BCrypt. Se han aplicado sentencias preparadas al 100% de las consultas SQL, validación de tipo MIME real en la subida de ficheros y política de contraseña fuerte.
- Objetivo 2 – Privacidad por diseño: cumplido. Los ecos son anónimos para el público. Los mensajes y todos sus datos asociados se eliminan en cascada. No hay mecanismos de seguimiento entre usuarios.
- Objetivo 3 – Experiencia de usuario de calidad: cumplido y superado. La puntuación de 99/100 en Rendimiento y 100/100 en Accesibilidad de Google Lighthouse supera los mínimos establecidos. La interfaz funciona correctamente en escritorio, tablet y móvil en todos los navegadores probados.
- Objetivo 4 – Moderación comunitaria: cumplido. El sistema de reportes notifica automáticamente al administrador al superar el umbral de 5 reportes. El panel de administración permite tres acciones diferenciadas.
- Objetivo 5 – Rendimiento y buenas prácticas: cumplido. Vanilla JS sin frameworks, Composer para dependencias, .env para credenciales.

7.1.2. Dificultades encontradas

Las principales dificultades técnicas encontradas durante el desarrollo, y cómo se resolvieron, fueron:

- Fallo catastrófico del hardware de almacenamiento: durante la fase de desarrollo, el disco duro mecánico donde se alojaba la documentación del TFG sufrió una corrupción grave de sectores, quedando completamente ilegible. Se intentó mitigar el problema mediante herramientas de clonación y recuperación forense en un entorno Linux (usando comandos de bajo nivel como dd y ddrescue), pero el daño físico del disco impidió la extracción de los datos. Como solución inevitable, se tuvo que rehacer la documentación desde cero, lo que supuso un impacto imprevisto en el proyecto.
- Sistema 2FA con PHPMailer: configurar correctamente el servidor SMTP de Gmail requirió investigar los requisitos de autenticación modernos de Google. Los correos llegaban inicialmente a la carpeta de spam. La solución fue usar STARTTLS (puerto 587) en lugar de SMTPS, generar una Contraseña de Aplicación específica de Google, añadir el campo AltBody con texto plano y configurar correctamente las cabeceras From. Esta dificultad generó la mayor desviación temporal respecto a la planificación.
- Sincronización de contadores de reacciones: la desnormalización de los contadores en la tabla mensajes requirió una gestión cuidadosa para evitar inconsistencias. La solución fue usar operaciones atómicas ($\text{tipo} = \text{tipo} + 1$ / $\text{tipo} = \text{tipo} - 1$) y desarrollar el script sync_reactions.php como herramienta de mantenimiento.
- Diseño responsivo del modal de mensaje en dispositivos móviles: el panel glassmorphism requirió ajustes de overflow, max-height y uso de dvh (dynamic viewport height) para Safari iOS, que interpreta las unidades vh de forma diferente al incluir la barra de herramientas del navegador en el cálculo.



7.1.3. Grado de satisfacción

El grado de satisfacción con el trabajo realizado es muy alto. ShootStars es una aplicación funcional, segura y visualmente atractiva que está desplegada en producción y accesible públicamente. La metáfora conceptual de las estrellas fugaces se ha trasladado con coherencia a todas las capas del proyecto: el nombre, la interfaz visual, los términos del dominio (Estrellas, Ecos Estelares) y la URL del proyecto.

El proyecto ha supuesto un aprendizaje real y profundo de todo el ciclo de desarrollo de una aplicación web: desde la administración del servidor Linux hasta la seguridad del backend, pasando por el diseño de bases de datos relacionales, la implementación de interfaces modernas con CSS avanzado y la integración de servicios externos como PHPMailer/SMTP.

7.2. Posibles Ampliaciones

A corto plazo (sin cambios arquitecturales)

- Cambio de reacción: actualmente `reaction.php` devuelve HTTP 409 Conflict si el usuario ya ha reaccionado. Sería sencillo añadir la lógica de cambio: si el usuario ya tiene una reacción activa para ese mensaje, actualizar el tipo en lugar de rechazar la operación.
- Expiración real de mensajes: añadir un campo `expires_at` DATETIME en la tabla `mensajes` y un script cron (cron job) que ejecute periódicamente `UPDATE mensajes SET visible=0 WHERE expires_at < NOW() AND expires_at IS NOT NULL`, dando a ShootStars una dimensión verdaderamente efímera.
- Rate limiting en el login: implementar un contador de intentos fallidos por IP/usuario y un bloqueo temporal (5 minutos) tras 5 intentos fallidos, previniendo ataques de fuerza bruta contra el sistema de autenticación.

A medio plazo (nuevas funcionalidades)

- Progressive Web App (PWA): añadir un Service Worker (fichero JavaScript que actúa como proxy de red) y un Web App Manifest (fichero JSON que describe la aplicación) para hacer ShootStars instalable en el escritorio o pantalla de inicio del móvil sin pasar por una tienda de aplicaciones.
- Moderación asistida por IA: integrar IA para analizar el texto de cada mensaje antes de publicarse y asignarle una puntuación de toxicidad. Los mensajes que superen un umbral configurable podrían requerir revisión manual antes de ser visibles en el feed.
- Categorías o etiquetas de mensajes: añadir al formulario de composición la posibilidad de elegir entre varias categorías temáticas (reflexiones, humor, desahogo, etc.), permitiendo a los usuarios filtrar por tipo de mensaje en el feed.

A largo plazo (cambios arquitecturales)

- API REST formal con JWT: separar completamente el backend (API RESTful documentada con OpenAPI/Swagger y autenticación mediante tokens JWT, JSON Web Token) del frontend, lo que permitiría desarrollar clientes nativos para Android e iOS que consuman la misma API.
- Migración a un framework PHP (Slim o Laravel): a medida que la aplicación crezca en funcionalidades, adoptar un framework con soporte nativo para enrutamiento, ORM (Eloquent en Laravel), middleware, testing automatizado (PHPUnit) y gestión de caché.
- Sistema de notificaciones en tiempo real con WebSockets: en lugar de la carga polling bajo demanda de los ecos, implementar WebSockets (mediante la librería Ratchet para PHP o un servicio externo como Pusher) para notificar en tiempo real al usuario cuando recibe un nuevo eco en alguno de sus mensajes.

8. BIBLIOGRAFÍA

Las fuentes de información utilizadas en la elaboración de este proyecto y memoria:

Documentación técnica oficial

PHP Group. (2024) PHP 8.2 Manual. Disponible en: <https://www.php.net/manual/es/>

Oracle Corporation / MariaDB Foundation. (2024) MySQL 8.0 Reference Manual; MariaDB Knowledge Base. Disponible en: <https://dev.mysql.com/doc/refman/8.0/en/> y <https://mariadb.com/kb/en/>

Mozilla Developer Network. (2024) MDN Web Docs: HTML, CSS y JavaScript Reference. Disponible en: <https://developer.mozilla.org/es/>

Apache Software Foundation. (2024) Apache HTTP Server 2.4 Documentation. Disponible en: <https://httpd.apache.org/docs/2.4/>

Bointon, M., & PHPMailer Contributors. (2024) PHPMailer v6.11. Disponible en: <https://github.com/PHPMailer/PHPMailer>

Composer. (2024) Composer: A Dependency Manager for PHP. Disponible en: <https://getcomposer.org/doc/>

Seguridad web

OWASP Foundation. (2021) OWASP Top Ten 2021. Disponible en: <https://owasp.org/Top10/>

OWASP Foundation. (2023) OWASP Testing Guide v4.2. Disponible en: <https://owasp.org/www-project-web-security-testing-guide/>

Diseño, accesibilidad y rendimiento web

W3C. (2018) Web Content Accessibility Guidelines (WCAG) 2.1. Disponible en: <https://www.w3.org/TR/WCAG21/>

Google Developers. (2024) Chrome Lighthouse: Automated auditing, performance metrics, and best practices for the web. Disponible en: <https://developer.chrome.com/docs/lighthouse/>

Google Developers. (2024) Core Web Vitals. Disponible en: <https://web.dev/vitals/>

Webgrafía

Stack Overflow. (2024) PHP, MySQL, JavaScript, CSS Communities. Disponible en: <https://stackoverflow.com/>

W3Schools. (2024) PHP Tutorial, MySQL Tutorial, JavaScript Tutorial, CSS Tutorial. Disponible en: <https://www.w3schools.com/>

DigitalOcean. (2024) How To Install Linux, Apache, MySQL, PHP (LAMP) Stack on Debian 12. Disponible en: <https://www.digitalocean.com/community/tutorials/>

CSS-Tricks. (2024) A Complete Guide to Flexbox; A Complete Guide to CSS Grid. Disponible en: <https://css-tricks.com/>

Google Fonts. (2024) Orbitron — Designed by Matt McInerney; Outfit — Designed by Rodrigo Fuenzalida. Disponible en: <https://fonts.google.com>

9. RELACIÓN DE FICHEROS EN FORMATO DIGITAL

A continuación se relacionan los ficheros que en formato digital se incorporan a la presente memoria. Todos ellos se adjuntan en el soporte digital..

Nombre del fichero/carpeta	Formato	Descripción
Memoria_ShootStars.docx / .pdf	DOCX / PDF	La presente memoria técnica del proyecto en formato editable y en formato PDF.
backend/	Carpeta	Código fuente PHP del servidor: módulo de autenticación, endpoints de la API interna (18 ficheros), herramientas de mantenimiento y configuración de Composer.
frontend/	Carpeta	Código fuente del cliente: 8 páginas PHP/HTML, hoja de estilos CSS única (main.css, ~998 líneas), scripts JavaScript (bg.js, msg.js) y recursos estáticos (logo, avatar por defecto).
database/schema.sql	SQL	Script DDL completo para la creación de la base de datos del proyecto. Incluye todas las tablas, claves foráneas, restricciones UNIQUE e índices.
database/test-values.sql	SQL	Datos de prueba para poblar la base de datos en un entorno de desarrollo y facilitar la evaluación de la aplicación.
.env.example	ENV	Plantilla del fichero de configuración de entorno. El fichero .env real (con credenciales) no se incluye por razones de seguridad.

Nombre del fichero/carpeta	Formato	Descripción
.htaccess	HTAcces s	Fichero de configuración de Apache para el enrutamiento por URL limpia y las restricciones de seguridad.
README.md	Markdow n	Documentación del repositorio en formato Markdown con instrucciones de instalación, tecnologías usadas y descripción del proyecto.
docs/Recomendaciones_Memoria.pdf	PDF	Documento oficial del IES María Moliner con las recomendaciones para la elaboración de la memoria de proyecto.
docs/Anteproyecto - Félix.pdf	PDF	Documento del anteproyecto presentado al inicio del módulo de Proyecto para la aprobación del tema.

El código fuente completo de la aplicación también está disponible públicamente en el repositorio de GitHub <https://github.com/darks1g/ShootStars> con historial completo de commits. La aplicación está desplegada y accesible en <https://shootstars.sytes.net>.

ANEXO I – GLOSARIO

Relación de términos relevantes del proyecto ordenados alfabéticamente con breve definición:

- 2FA (Two-Factor Authentication / Autenticación de Doble Factor): mecanismo de seguridad que requiere dos tipos de verificación para autenticar a un usuario. En ShootStars: contraseña + código OTP enviado al correo electrónico.
- AJAX (Asynchronous JavaScript And XML): técnica de desarrollo web que permite actualizar partes de una página sin recargarla completamente, mediante peticiones HTTP asíncronas desde JavaScript.
- API (Application Programming Interface): interfaz que define cómo dos sistemas de software se comunican entre sí. En ShootStars, la API interna son los endpoints PHP que el frontend consulta mediante Fetch.
- BCrypt: algoritmo de hashing criptográfico diseñado para contraseñas. Incluye un factor de coste (cost factor) que lo hace deliberadamente lento para dificultar ataques de fuerza bruta. Es el algoritmo seleccionado por PASSWORD_DEFAULT en PHP.

- Canvas (HTML5 Canvas API): elemento HTML5 que proporciona un área de dibujo bidimensional controlable mediante JavaScript. ShootStars lo utiliza para el sistema de partículas estelares del fondo.
- Cascade (SQL ON DELETE CASCADE): comportamiento de una clave foránea que propaga automáticamente la eliminación de un registro padre a todos los registros hijo relacionados.
- COCOMO (Constructive Cost Model): modelo de estimación del esfuerzo y costo de proyectos de software desarrollado por Barry Boehm en 1981, basado en el número de líneas de código fuente.
- Composer: gestor de dependencias estándar para PHP. Permite declarar, instalar y actualizar las librerías externas de un proyecto mediante el fichero composer.json.
- CSS3 (Cascading Style Sheets nivel 3): lenguaje de presentación que define el aspecto visual de las páginas web. ShootStars utiliza variables custom, animaciones @keyframes, flexbox, glassmorphism y gradientes radiales.
- DDNS (Dynamic DNS): servicio que actualiza automáticamente un nombre de dominio cuando cambia la dirección IP del servidor. ShootStars usa No-IP para mantener el dominio shootstars.sytes.net.
- Desnormalización: decisión de diseño de bases de datos que viola intencionadamente las formas normales de Codd para mejorar el rendimiento de lectura, a costa de redundancia controlada de datos.
- DOM (Document Object Model): representación en árbol de los elementos HTML de una página web, manipulable desde JavaScript para añadir, eliminar o modificar contenido dinámicamente.
- ECO (Eco Estelar): término propio del dominio de ShootStars para las respuestas anónimas enviadas a un mensaje (estrella). Máximo 150 caracteres.
- Estrella: término propio del dominio de ShootStars para los mensajes publicados por los usuarios. Máximo 500 caracteres. Se distribuyen aleatoriamente entre todos los visitantes.

- Fetch API: interfaz nativa del navegador para realizar peticiones HTTP asíncronas desde JavaScript, basada en Promesas. Reemplaza al antiguo XMLHttpRequest.
- finfo (PHP FileInfo): extensión de PHP que analiza los "magic bytes" (primeros bytes de cabecera) de un fichero para determinar su tipo MIME real, independientemente de la extensión del nombre del fichero.
- Glassmorphism: tendencia de diseño UI que simula el efecto de cristal esmerilado mediante backdrop-filter: blur() combinado con fondo semitransparente.
- Hash / Hashing: proceso matemático unidireccional que transforma una entrada (contraseña) en una cadena de caracteres de longitud fija. No es reversible: no se puede obtener la contraseña original a partir del hash.
- InnoDB: motor de almacenamiento de MySQL/MariaDB con soporte de transacciones ACID, claves foráneas con CASCADE y bloqueo a nivel de fila.
- LAMP: acrónimo de Linux + Apache + MySQL + PHP. Stack tecnológico de servidor web de código abierto muy extendido.
- LOC (Lines of Code): medida de tamaño de software basada en el número de líneas de código fuente. Utilizada como entrada del modelo COCOMO.
- MIME (Multipurpose Internet Mail Extensions): estándar que define los tipos de contenido de los ficheros en internet. Ejemplo: image/jpeg, text/html, application/json.
- mod_rewrite: módulo de Apache que permite reescribir internamente las URLs sin redirección al navegador, mediante reglas definidas en ficheros .htaccess.
- OTP (One-Time Password): contraseña de un solo uso, válida únicamente para una transacción o sesión de autenticación específica. En ShootStars, un número de 6 dígitos generado aleatoriamente.
- OWASP (Open Web Application Security Project): fundación sin ánimo de lucro dedicada a mejorar la seguridad del software web. Su OWASP Top 10 enumera las vulnerabilidades más críticas.
- PHPMailer: librería PHP para el envío de correos electrónicos con soporte SMTP, TLS, HTML y archivos adjuntos. La única dependencia externa del proyecto.

- Prepared Statement (Sentencia Preparada): mecanismo de las APIs de bases de datos que separa el código SQL de los datos del usuario, previniendo la inyección SQL al nivel del protocolo de comunicación con la base de datos.
- RGPD (Reglamento General de Protección de Datos): reglamento de la Unión Europea (2016/679) que regula el tratamiento de datos personales y reconoce el derecho al olvido (artículo 17).
- SMTP (Simple Mail Transfer Protocol): protocolo estándar de internet para la transmisión de correos electrónicos entre servidores, definido en el RFC 5321.
- STARTTLS: mecanismo de negociación de cifrado en SMTP que eleva una conexión inicialmente no cifrada (puerto 587) a una conexión cifrada TLS sin necesidad de un puerto separado.
- Vanilla JavaScript: término coloquial para referirse al uso de JavaScript puro, sin bibliotecas ni frameworks externos como jQuery, React, Vue o Angular.
- XSS (Cross-Site Scripting): vulnerabilidad de seguridad web que permite a un atacante inyectar código JavaScript malicioso en páginas vistas por otros usuarios. Se previene con `htmlspecialchars()` en PHP y `textContent` en JavaScript.
- utf8mb4: juego de caracteres de MySQL que soporta el rango completo de Unicode, incluyendo emojis (caracteres de 4 bytes). Necesario para el correcto almacenamiento de emojis en la base de datos.

ANEXO II – HOJA DE ESTILOS CSS (`main.css`): EXTRACTOS REPRESENTATIVOS

A continuación se reproducen los extractos más significativos del fichero `frontend/css/main.css` (~998 líneas), que define la totalidad del sistema visual de ShootStars. Se incluye este anexo para documentar las decisiones de diseño CSS más reseñables y el sistema de animaciones implementado.

A2.1. Reset y Fondo Global

```
/*
 * SHOOTSTARS - MAIN CSS
 * Organized, Modular and Performant Space Aesthetics
 */

@import url("https://fonts.googleapis.com/css2?
    family=Outfit:wght@300;400;700
    &family=Orbitron:wght@400;500;700&display=swap");

* { box-sizing: border-box; }

html, body {
    background-color: #050014;
    background: radial-gradient(
        circle at bottom center,
        #240046 0%,      /* morado oscuro en el centro
inferior */
        #10002b 40%,    /* morado muy oscuro */
        #000000 100%    /* negro en los extremos */
    );
    margin: 0; padding: 0;
    overflow: hidden; overflow-x: hidden;
    width: 100%; height: 100%;
    font-family: "Outfit", sans-serif;
    color: white;
}

/* Canvas de estrellas: ocupa toda la pantalla detrás de
todo */
canvas {
    display: block;
    position: fixed;
    top: 0; left: 0;
    z-index: 0;
}

/* Efecto fade-in del canvas al cargar la página */
#space { opacity: 0; transition: opacity 1.5s ease-in; }
#space.fade-in { opacity: 1; }
```


A2.2. Sistema de Animaciones @keyframes

```
/* Animación de brillo pulsante para el logo/título principal */
@keyframes glow {
  from { text-shadow: 0 0 20px #7209b7, 0 0 40px #4361ee; }
  to   { text-shadow: 0 0 30px #f72585, 0 0 60px #7209b7; }
}

/* Animación de brillo suave para el nombre del usuario en correos */
@keyframes pulseGlow {
  from { text-shadow: 0 0 5px #f72585; }
  to   { text-shadow: 0 0 15px #f72585, 0 0 25px #7209b7; }
}

/* Animación de entrada de los ecos (slide desde abajo con desvanecimiento) */
@keyframes ecoSlideIn {
  from { opacity: 0; transform: translateY(10px); }
  to   { opacity: 1; transform: translateY(0); }
}

/* Aplicación de la animación de entrada a cada eco */
.eco-item { animation: ecoSlideIn 0.3s ease-out forwards; }
```

A2.3. Sistema de Botones

ShootStars define un sistema de botones unificado mediante clases CSS que garantizan coherencia visual. Todos los botones comparten una base común y se especializan mediante clases adicionales:

```
/* Base compartida por todos los botones */
.btn-primary, .btn-admin, .btn-edit,
.btn-delete, .btn-save, .btn-eco {
  font-family: "Outfit", sans-serif;
  font-weight: 700;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275);
  text-transform: uppercase;
}
```

```

    letter-spacing: 1px;
    position: relative;
    overflow: hidden;
}

/* Botón primario: gradiente cian → azul eléctrico */
.btn-primary {
    background: linear-gradient(135deg, #4cc9f0 0%, #4361ee
100%);
    color: white;
    padding: 12px 24px;
    box-shadow: 0 0 15px rgba(67, 97, 238, 0.5);
}
.btn-primary:hover {
    transform: translateY(-2px);
    box-shadow: 0 0 25px rgba(67, 97, 238, 0.8);
}

/* Botón de navegación: subrayado animado al hacer hover */
.btn-nav {
    color: #fff; text-decoration: none;
    font-weight: 500; position: relative;
    padding: 5px 0; transition: color 0.3s;
}
.btn-nav::after {
    content: ""; position: absolute;
    width: 0; height: 2px;
    bottom: 0; left: 0;
    background-color: #4cc9f0;
    transition: width 0.3s;
}
.btn-nav:hover::after { width: 100%; }
.btn-nav:hover { color: #4cc9f0; }

/* Botón eliminar: gradiente rojo */
.btn-delete {
    background: linear-gradient(135deg, #e63946 0%, #d62828
100%) !important;
    color: #000 !important;
    box-shadow: 0 4px 10px rgba(230, 57, 70, 0.3);
}
.btn-delete:hover {
    transform: translateY(-2px) scale(1.02);
    box-shadow: 0 6px 15px rgba(230, 57, 70, 0.5);
    color: white !important;
}

```

A2.4. Responsividad (Media Queries)

El diseño responsivo se implementa mediante media queries con un punto de corte principal en 768px (separación entre móvil y escritorio). En dispositivos móviles, los elementos se reordenan en columna y el panel glassmorphism del mensaje se ajusta al viewport:

```
/* === RESPONSIVIDAD - Breakpoint móvil: 768px === */
@media (max-width: 768px) {

  /* Cabecera: de horizontal a vertical */
  .main-header {
    padding: 10px 15px;
    flex-direction: column;
    gap: 10px;
    background: rgba(5, 0, 20, 0.9);
    backdrop-filter: blur(10px);
  }

  /* Título hero: reducir tamaño en móvil */
  .hero-title {
    font-size: 3rem;
    letter-spacing: 2px;
  }

  /* Modal de mensaje: ancho fluido y altura limitada */
  .mensaje-box {
    width: 88% !important;
    max-height: 70vh !important;
    margin: auto !important;
  }

  /* Área de scroll del modal con padding extra para botones fijos */
  .modal-scroll-area {
    padding: 55px 20px 20px !important;
  }

  /* Reacciones: botones más grandes para pantallas táctiles */
  .reaccion {
    font-size: 1.2rem !important;
    padding: 8px 12px !important;
  }

  /* Dashboard y Admin: reducir padding superior */
```

```

.dashboard-container, .admin-container {
    padding: 160px 15px 50px;
}

/* Formularios de autenticación: ancho fluido */
.auth-box {
    width: 90%;
    max-width: 400px;
    padding: 25px;
}

```

A2.5. Sistema de Formularios con Validación Visual

Los formularios de la aplicación utilizan clases CSS para proporcionar feedback visual inmediato de la validación. El estado de cada campo se indica mediante cambio de color del borde y texto de ayuda:

```

/* Campos de formulario con transición suave de borde */
.form-group input {
    width: 100%;
    padding: 12px;
    border: 1px solid rgba(255, 255, 255, 0.15);
    border-radius: 8px;
    background: rgba(0, 0, 0, 0.2);
    color: white;
    font-family: "Outfit", sans-serif;
    transition: all 0.3s ease;
}

/* Estado focus: borde cian con halo luminoso */
.form-group input:focus {
    outline: none;
    border-color: #4cc9f0;
    box-shadow: 0 0 15px rgba(76, 201, 240, 0.3);
    background: rgba(0, 0, 0, 0.4);
}

/* Mensajes de error/éxito en formularios */
.error-msg {
    background: rgba(230, 57, 70, 0.2);
    border: 1px solid rgba(230, 57, 70, 0.5);
    color: #ffadad;
}

```

```

/* Scrollbars personalizados (WebKit: Chrome, Safari, Opera)
*/
::-webkit-scrollbar { width: 6px; height: 6px; }
::-webkit-scrollbar-thumb {
    background: #4cc9f0;
    border-radius: 10px;
}
/* Color diferente por sección */
#reports-list::-webkit-scrollbar-thumb { background:
#e63946; }
.ecos-container::-webkit-scrollbar-thumb { background:
#7209b7; }

/* Firefox: propiedad scrollbar-color */
#reports-list { scrollbar-color: #e63946
rgba(255,255,255,0.05); }
#messages-list { scrollbar-color: #4cc9f0
rgba(255,255,255,0.05); }

```

ANEXO III – VALIDACIÓN DEL FORMULARIO DE REGISTRO

El formulario de registro de ShootStars implementa una experiencia de usuario avanzada con validación en tiempo real mediante peticiones AJAX. A continuación se reproduce el código JavaScript embebido en register.php que gestiona este comportamiento, por ser uno de los fragmentos más elaborados de la interfaz:

```

/* Script embebido en frontend/register.php */

const userInput  = document.getElementById("username");
const emailInput = document.getElementById("email");
const passInput  = document.getElementById("password");
const confInput  =
document.getElementById("password_confirm");
const btnSubmit  =
document.querySelector("button[type=submit]");

// Estado de validación de cada campo
let userValid  = false;
let emailValid = false;
let passStrong = false;
let passMatch  = false;

// Actualiza el estado del botón de envío

```

```

// Solo se activa cuando los 4 campos son válidos
simultáneamente
function updateBtn() {
    const allValid = userValid && emailValid && passStrong
&& passMatch;
    btnSubmit.disabled      = !allValid;
    btnSubmit.style.opacity = allValid ? "1" : "0.5";
}

// Helper para actualizar los mensajes de estado visual
function setStatus(el, text, isGood) {
    el.textContent = text;
    el.style.color = isGood ? "#4cc9f0" : "#e63946";
}

// Función genérica de verificación AJAX contra el backend
function checkBackend(field, value, msgEl, callback) {
    const fd = new FormData();
    fd.append("field", field);
    fd.append("value", value);
    fetch("/backend/auth/check_user.php", { method: "POST",
body: fd })
        .then(r => r.json())
        .then(data => callback(data.exists));
}

// Verificación de nombre de usuario al perder el foco
userInput.addEventListener("blur", () => {
    const val = userInput.value.trim();
    if (val.length < 3) {
        setStatus(msgUser, "✖ Mínimo 3 caracteres", false);
        userValid = false;
        updateBtn();
        return;
    }
    checkBackend("username", val, msgUser, (exists) => {
        userValid = !exists;
        setStatus(msgUser,
            exists ? "✖ El usuario ya existe"
                : "✓ Usuario disponible",
            !exists);
        updateBtn();
    });
});

// Verificación de email al perder el foco
emailInput.addEventListener("blur", () => {

```

```

    const val = emailInput.value.trim();
    if (!val.includes("@")) {
        setStatus(msgEmail, "✖ Email no válido", false);
        emailValid = false;
        updateBtn();
        return;
    }
    checkBackend("email", val, msgEmail, (exists) => {
        emailValid = !exists;
        setStatus(msgEmail,
            exists ? "✖ Email ya registrado"
                : "✔ Email disponible",
            !exists);
        updateBtn();
    });
});

// Misma expresión regular que el backend PHP para
coherencia
const strongRegex =
/^((?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,})$/;

// Validación de fortaleza de contraseña en tiempo real
(keyup)
passInput.addEventListener("keyup", () => {
    passStrong = strongRegex.test(passInput.value);
    setStatus(msgStrength,
        passStrong
            ? "✔ Contraseña fuerte"
            : "✖ 8+ chars, Mayúsculas, Minúsculas, Números
y Símbolo",
        passStrong);
    checkMatch(); // Verificar también la confirmación
});

// Verificación de coincidencia de contraseñas
function checkMatch() {
    passMatch = (passInput.value === confInput.value
        && passInput.value !== "");
    setStatus(msgMatch,
        passMatch ? "✔ Las contraseñas coinciden"
            : "✖ Las contraseñas no coinciden",
        passMatch);
    updateBtn();
}
confInput.addEventListener("keyup", checkMatch);

```

```
// Estado inicial: botón desactivado  
btnSubmit.disabled = true;  
btnSubmit.style.opacity = "0.5";
```

Este fragmento muestra cómo la validación en el frontend replica exactamente las mismas reglas que el backend PHP (misma expresión regular, mismos endpoints de verificación AJAX), garantizando coherencia entre ambas capas y proporcionando al usuario un feedback inmediato sin necesidad de enviar el formulario.